PEOPLE'S DEMOCRATIC REPUBLIC OF ALGERIA

Ministry of Higher Education and Scientific Research

University 20 Août 1955 – Skikda

Faculty of Sciences

Department of Computer Science

Master's Thesis for the Degree of **Academic Master in Computer Science**

Option: ***Advanced Software Engineering and Applications***

Under the framework of Ministerial Decree No. 1275 dated September 27, 2022, defining the modalities of preparing a graduation project for obtaining a university degree linked to the creation of an innovative enterprise (startup).

Design and Implementation of a Medical Diagnostic Support Platform Based on Multimodal AI via the Cloud and Deployment on Mobile Platform: Case of Skin Cancers

Presented by:

- **LAIDI SISTA Abderrahmane**
- **CHAHOUD Oussama**

Supervised by:

- **Dr. KERRAOUI Sara**

Academic Year 2025/2026

Abstract / Résumé / ملخص

Abstract

This Master's thesis presents the design, engineering, and experimental evaluation of **MedAssist AI**, an end-to-end cloud and mobile clinical decision support platform for **skin lesion classification** powered by multimodal deep learning. Addressing the global scarcity of dermatologists and bridging the gap between academic ML research and operational healthcare deployment, we propose a *Gated Cross-Attention Fusion* neural architecture combining an *EfficientNet-B3* visual encoder with Generalized Mean (GeM) pooling and a Multi-Layer Perceptron (MLP) branch for seven standardized patient clinical metadata features. Trained on a unified multi-center dataset of 6,581 images (PAD-UFES-20, ISIC 2019, MCR-SL) under strict patient-level zero-leakage stratification, the final V6.0 model achieves an accuracy of **74.31%** and a **Macro F1-Score of 0.7315** on the benchmark test set ($n = 654$), outperforming pure visual unimodal classification by **+11.65 Macro F1 points**. The complete production-grade software ecosystem couples a Docker-containerized FastAPI backend service exhibiting a measured nominal latency of **1,470 ms (p_{95})**, an ergonomic Flutter mobile application, and an automated clinical risk escalation engine.

Keywords: Skin Lesion Classification, Multimodal Deep Learning, Gated Cross-Attention, EfficientNet-B3, FastAPI, Flutter, Telemedicine, MLOps.

Résumé

Ce mémoire de fin d'études présente la conception, le développement et la validation expérimentale approfondie de **MedAssist AI**, une plateforme cloud et mobile d'aide à la décision clinique pour la **classification des lésions cutanées** par apprentissage profond multimodal. Afin de surmonter le déficit d'accès au dépistage précoce et de combler le fossé structurel entre recherche algorithmique et déploiement opérationnel en télémédecine, nous proposons une architecture neuronale basée sur un mécanisme de fusion par attention croisée contrôlée (*Gated Cross-Attention Fusion*) couplant l'encodeur visuel *EfficientNet-B3* doté de pooling GeM (*Generalized Mean*) et une branche MLP dédiée à sept variables cliniques standardisées. Entraîné sur un jeu de données unifié de 6 581 images multi-sources (PAD-UFES-20, ISIC 2019, MCR-SL) sous partitionnement stratifié strict au niveau patient sans fuite d'information (*zero-leakage*), le modèle final V6.0 délivre une exactitude globale de **74,31 %** et un score **Macro F1 de 0,7315** sur le jeu de test indépendant ($n = 654$), surpassant l'approche unimodale visuelle de **+11,65 points de Macro F1**. L'écosystème logiciel intègre un service backend REST FastAPI conteneurisé sous Docker présentant une latence nominale mesurée de **1 470 ms (p_{95})**, une application mobile Flutter ergonomique avec capture guidée, et un moteur automatisé d'escalade du risque clinique.

Mots-clés : Classification des lésions cutanées, Apprentissage profond multimodal, Gated Cross-Attention, EfficientNet-B3, FastAPI, Flutter, Télémédecine, MLOps.

ملخص

تتناول هذه المذكرة تصميم وهندسة والتقييم التجريبي لمنصة MedAssist AI، وهي منصة سحابية ومحمولة متكاملة لدعم القرار السريري في **تصنيف الآفات الجلدية** باستخدام التعلم العميق متعدد الوسائط. ولمعالجة النقص العالمي في أخصائيي الجلدية وسد الفجوة بين الأبحاث الأكاديمية والتطبيق السريري في الطب الاتصالي، نقترح معمارية عصبية قائمة على آلية الانتباه المتقاطع الموجه (Gated Cross-Attention Fusion) تجمع بين المشفر البصري EfficientNet-B3 مع تجميع المتوسط العام المعمم (GeM Pooling) وفرع شبكة متعددة الطبقات (MLP) لسبعة متغيرات سريرية قياسية للمريض. بعد التدريب على مجموعة بيانات موحدة متعددة المصادر تضم 6,581 صورة (PAD-UFES-20، ISIC 2019، MCR-SL) تحت تقسيم طبقي صارم على مستوى المريض يمنع تسرب المعلومات كلياً، حقق النموذج النهائي V6.0 دقة إجمالية قدرها **74.31%** ودرجة **Macro F1** بلغت **0.7315** على مجموعة الاختبار المستقلة (n=654)، متفوقاً على النموذج البصري أحادي الوسيط بفارق **11.65+ نقطة في مقياس Macro F1**. كما يتكامل النظام مع بيئة برمجية جاهزة للنشر تضم خادم FastAPI خلفياً وحاوية Docker بزمن استجابة اسمي قدره **1,470 ms (p95)**، وتطبيقاً محمولاً عبر Flutter مع التقاط موجه، ومحركاً مؤتمتاً لتصعيد مستوى المخاطر السريرية.

الكلمات المفتاحية: تصنيف الآفات الجلدية، التعلم العميق متعدد الوسائط، Gated Cross-Attention، EfficientNet-B3، FastAPI، Flutter، الطب الاتصالي، MLOps.

Acknowledgements

First and foremost, we express our profound gratitude to **Allah** the Almighty for granting us the health, perseverance, and determination required to bring this academic work to completion.

We extend our deepest and most sincere appreciation to our thesis supervisor, **Dr. KERRAOUI Sara**, for her exceptional guidance, continuous availability, and insightful advice throughout the preparation of this dissertation. Her scientific rigor, constructive feedback, and unwavering support were instrumental in the successful realization of this project.

We also wish to express our heartfelt gratitude to the distinguished members of the **Jury** for taking the time to evaluate this thesis and for honoring us with their valuable observations and critical recommendations.

Our warm thanks go to the entire **faculty and teaching staff** of the Department of Computer Science at the University of 20 Août 1955 – Skikda, and particularly to the professors of the *Advanced Software Engineering and Applications (GLAA)* program, for the high standard of academic education provided throughout our curriculum.

Finally, we express our sincere appreciation to everyone who contributed directly or indirectly to the success of this work: our families for their endless patience and unconditional encouragement, and our peers for their constructive discussions and companionship.

LAIDI SISTA Abderrahmane

CHAHOUD Oussama

Dedications

LAIDI SISTA Abderrahmane

*I dedicate this modest work to my beloved **parents**, for the priceless sacrifices they have made on my behalf, for their endless patience, their unconditional love, and their steadfast trust in me. No dedication could ever adequately convey the depth of my gratitude. May the Almighty grant them good health, serenity, and long life.*

*To my **brothers and sisters**, for their constant encouragement and unwavering support throughout every milestone of this academic journey.*

*To all my **friends and colleagues**, who shared moments of doubt, perseverance, and joy throughout these years of study.*

*To all the **teachers and mentors** who have guided and inspired me, from the very beginning of my education to this day.*

CHAHOUD Oussama

*I dedicate this work to my **parents**, an enduring source of strength and inspiration, for their boundless devotion, guidance, and prayers. May they find in this modest thesis a sincere testament to my deep affection and eternal gratitude.*

*To my entire **family**, for believing in me and standing by my side at every step of this journey.*

To all those who have supported me near and far over the years, and who have contributed in their own meaningful way to making me who I am today.

Contents

Abstract / Multilingual Summary i

Acknowledgements iii

Dedications iv

List of Acronyms and Abbreviations xii

1 General Introduction 1

1.1 Context and Motivation . . . 1
1.2 Problem Statement . . . 1
1.2.1 Limitations of Purely Visual Approaches . . . 2
1.2.2 The Performance–Deployment Paradox . . . 2
1.2.3 Absence of a Unified Software Architecture . . . 2
1.3 Proposed Solution: MedAssist AI . . . 2
1.4 Objectives and Contributions . . . 3
1.4.1 General Objective . . . 3
1.4.2 Specific Objectives . . . 3
1.4.3 Contributions . . . 3
1.5 Research Questions . . . 4
1.6 Organization of the Dissertation . . . 4

2 State of the Art 5

2.1 Introduction . . . 5
2.2 PRELUDE: From Traditional Medicine to AI . . . 5
2.2.1 The Traditional Diagnostic Paradigm . . . 5
2.2.2 The Emergence of Artificial Intelligence in Medicine . . . 5
2.2.3 From Opportunity to Reality: First Steps . . . 6
2.3 Foundations of Deep Learning . . . 7
2.3.1 Convolutional Neural Network Architectures . . . 7
2.3.2 Transfer Learning and Fine-Tuning in Medical Imaging . . . 9
2.3.3 Initial Applications in Dermatology . . . 10

2.4 Multimodal Datasets for Dermatology . . . 12
2.4.1 Context and Motivation . . . 12
2.4.2 HAM10000: First Large-Scale Multimodal Dataset . . . 12
2.4.3 PAD-UFES-20: Telemedicine-Oriented Dataset . . . 14
2.4.4 ISIC 2019: Benchmark Dataset for Multi-Class Classification . . . 17
2.4.5 MCR-SL: Multi-Ethnic Clinical Dataset . . . 18
2.4.6 Critical Comparison of the Four Datasets . . . 19
2.5 Multimodal Architectures — Evolution and Performance . . . 20
2.5.1 Principles of Multimodal Fusion . . . 20
2.5.2 Chronological Evolution (2018–2022) . . . 21
2.5.3 Synthesis: The Performance–Deployability Paradox . . . 22
2.6 The Deployment Challenge — From Research to Practice . . . 23
2.6.1 Introduction: The Research-to-Production Gap . . . 23
2.6.2 Edge Computing vs. Cloud Computing: Architectural Trade-off 23
2.6.3 Case Study: Oztel et al. (2023) — Mobile Deployment Attempt 25
2.6.4 Synthesis: The Multimodal–Deployability Paradox . . . 26
2.6.5 Comparative Synthesis of Related Work . . . 27
2.6.6 Formal Formulation of the Research Gap . . . 28
2.6.7 Positioning of MedAssist AI . . . 29
2.7 Chapter Conclusion . . . 30

3 MedAssist AI System Design 32
3.1 Introduction . . . 32
3.2 System Requirements . . . 32
3.2.1 Functional Requirements . . . 32
3.2.2 Non-Functional Requirements . . . 33
3.3 Architectural Design . . . 34
3.3.1 Use Case Diagram . . . 34
3.3.2 Three-Tier Architecture: Component View . . . 35
3.3.3 Sequence Diagram: Request Flow . . . 36
3.3.4 Class Diagram: Domain Model . . . 37
3.4 Data Preparation . . . 38
3.4.1 Class Distribution and Patient-Level Split . . . 38
3.4.2 Preprocessing Pipeline and Metadata Masking . . . 39
3.5 Multimodal Model Architecture (V6.0) . . . 40
3.5.1 Visual Branch: EfficientNet-B3, Projection, and Auxiliary Head 40
3.5.2 Clinical Branch: MLP with Masking and LayerNorm . . . 41
3.5.3 Gated Cross-Attention Fusion, Classifier, and Training Protocol 41
3.6 REST API Specification . . . 42

3.7 MLOps Pipeline, CI/CD, and Traceability . . . 43
3.8 Evaluation Protocol . . . 44
3.9 Conclusion . . . 44

4 Technical Implementation 46
4.1 Introduction . . . 46
4.2 Development Environment . . . 46
4.2.1 Hardware Environment . . . 46
4.2.2 Software Environment . . . 47
4.3 Modular Project Organization and Environments . . . 48
4.3.1 AI Pipeline Organization and Cloud Persistence (Google Drive / Colab) . . . 48
4.3.2 Backend Inference Server Architecture (FastAPI) . . . 48
4.3.3 Mobile Application Architecture (Flutter) . . . 49
4.4 AI Model Presentation and Mathematical Formalization . . . 51
4.4.1 Preprocessing Pipeline and Digital Normalization . . . 51
4.4.2 Clinical Metadata Branch and Availability Masking (`meta_mask`) 51
4.4.3 Visual Branch and Generalized Mean Pooling (GeM) . . . 52
4.4.4 Gated Cross-Attention Fusion Architecture . . . 53
4.4.5 Regularization Functions and Test-Time Augmentation Inference 54
4.4.6 Risk Assessment and Clinical Triage Module (`risk_assessment.py`) 54
4.4.7 Training Trajectories and Convergence Curves . . . 55
4.4.8 Confusion Matrix . . . 55
4.5 REST API Presentation . . . 56
4.5.1 Swagger UI Interface . . . 56
4.5.2 JSON Response Structure . . . 56
4.6 Mobile Application Presentation . . . 57
4.6.1 Home Screen . . . 57
4.6.2 Guided Acquisition Screen . . . 58
4.6.3 Clinical Form Screen . . . 58
4.6.4 Results Screen . . . 59
4.7 Testing and Validation . . . 60
4.7.1 Functional Testing . . . 60
4.7.2 Theoretical Capacity Analysis . . . 60
4.8 Conclusion . . . 61

5 Experimental Results and Evaluation 62
5.1 Introduction . . . 62
5.2 Data Engineering and V6.0 Experimental Protocol . . . 63
5.2.1 Pre-Split Merging and Multi-Source Data Harmonization . . . 63

5.2.2 Patient-Level Stratified Partitioning 63
5.2.3 Targeted Resampling and Smart Class Weights 64
5.3 Global and Detailed Diagnostic Performance of MedAssist V6.0 65
5.3.1 Training Dynamics and Convergence 65
5.3.2 Global Metrics and Objective Validation 65
5.3.3 Detailed Classification Report by Pathology 65
5.4 Ablation Study and Architectural Evolution 67
5.4.1 Version Trajectory and Evolutionary History (V1.0 to V6.0) . . 67
5.4.2 Controlled Multimodal Component Ablation Study 68
5.5 Software System Performance and Operational Validation 68
5.5.1 End-to-End Latency of the Inference Server 68
5.5.2 Theoretical Capacity and Scalability Analysis 69
5.5.3 Resource Utilization and Mobile Footprint 70
5.6 Discussion and Comparative Synthesis 71
5.6.1 Final Validation of Hypotheses 71
5.6.2 Positioning Against the State of the Art 71
5.7 Conclusion 72

6 General Conclusion and Future Perspectives 73
6.1 Synthesis of Completed Work and Evolutionary Trajectory 73
6.1.1 Engineering Trajectory and Evolution (V1.0–V6.0) 73
6.2 Validation of Hypotheses and Major Contributions 75
6.3 Identified Limitations and Challenges 75
6.4 Research Perspectives: Hybrid CNN-Transformer Encoder 76
6.4.1 Theoretical Foundations of the Hybrid Architecture 76
6.4.2 Expected Performance Projections 77
6.4.3 Technical Challenges and Mitigation Strategies 77
6.5 Roadmap for Clinical Deployment 78
6.5.1 Phase I — Hospital Pilot Study (6 to 12 Months) 78
6.5.2 Phase II — Explainability and FHIR Interoperability (12 to 18 Months) 78
6.5.3 Phase III — Scalability and Edge Inference (18 to 24 Months) . 78
6.5.4 Phase IV — Medical Certification and Regulatory Compliance (24 to 36 Months) 78
6.6 Final Conclusion and Closing Remarks 79

Bibliography 80

List of Figures

2.1 Classic architecture of a convolutional neural network 8

3.1 MedAssist AI use case diagram 35
3.2 Three-tier software architecture 36
3.3 Sequence diagram: Diagnostic request flow 37
3.4 Class diagram: MedAssist AI system 38

4.1 Implemented Gated Cross-Attention neural architecture (V6.0) 53
4.2 Training curves – Loss and Accuracy (Train / Validation) 55
4.3 Confusion matrix – MedAssist V6.0 model ($n = 654$, Macro F1: 0.7315) 56
4.4 Swagger UI interface – MedAssist AI API endpoints 56
4.5 Home Screen – Flutter Mobile Application 57
4.6 Guided Acquisition Screen – Circular centering overlay 58
4.7 Clinical Form Screen – Input of the 7 clinical metadata variables 58
4.8 Results Screen – Color-coded clinical risk and probabilities 59
4.9 Theoretical capacity analysis – Estimated p_{95} latency vs. concurrent users 60

5.1 Training dynamics – Loss and Macro F1 convergence (MedAssist V6.0) 65
5.2 Latency percentile distribution of the REST API (over 500 requests) . 69

List of Tables

1.1 Specific objectives and expected deliverables 3

2.1 Class distribution in HAM10000 13
2.2 Clinical metadata in PAD-UFES-20 15
2.3 Class distribution in PAD-UFES-20 16
2.4 Diagnostic class distribution in the ISIC 2019 training set 18
2.5 Comparison of the four datasets used in MedAssist AI 19
2.6 Comparison of multimodal architectures (performance vs. complexity) . 22
2.7 Comparative synthesis between Cloud and Edge Computing for mobile medical AI 25
2.8 Synthesis of the four dimensions of the gap between academic research and operational deployment 26
2.9 Comparative synthesis of AI-based dermatological diagnostic approaches 27

3.1 MedAssist AI functional requirements 33
3.2 MedAssist AI non-functional requirements 34
3.3 Diagnostic class distribution in the unified MedAssist AI dataset (V6.0) 39
3.4 Clinical variables selected for the model and presence mask (`meta_mask`) 40
3.5 Two-phase progressive training protocol (V6.0) 42
3.6 REST API endpoint specifications 42
3.7 Decision matrix and clinical risk level mapping 43
3.8 Target metrics and validated results on the unified benchmark (V6.0) . 44
3.9 Software system evaluation metrics 44

4.1 Hardware specifications across local, cloud, and mobile environments . 46
4.2 Modular decomposition matrix and software responsibilities 50
4.3 JSON response structure – POST /predict endpoint 57
4.4 Functional test suite execution results – pytest 60

5.1 Composition and distribution of the unified master dataset (6,581 images) 63
5.2 Patient-level stratified partitioning (Zero data leakage validation) . . . 64
5.3 Final per-class distribution across training, validation, and test sets . . 64
5.4 Global performance of the MedAssist V6.0 model (Test set, $n = 654$) . 65

5.5 Classification report by pathology – MedAssist V6.0 ($n = 654$) 66
5.6 Architectural evolution trajectory and empirical metrics (V1.0 to V6.0) 67
5.7 Component ablation study on the unified benchmark ($n = 654$) 68
5.8 Theoretical load capacity projection ($M/D/1$ analytical model) 70
5.9 Hardware footprint and system resources 70
5.10 Positioning of MedAssist AI V6.0 relative to the state of the art 71

6.1 Synthetic evolution matrix of the MedAssist AI system (V1.0 to V6.0) 74
6.2 Specification of the projected Dual-Encoder hybrid architecture 77
6.3 Expected performance projections of the hybrid model (Test set $n = 654$) 77

List of Acronyms and Abbreviations

Acronym	Definition and Description
ACK	Actinic Keratosis (Pre-cancerous epidermal lesion induced by chronic UV exposure)
API	Application Programming Interface
AUC	Area Under the ROC Curve (Diagnostic performance metric)
BCC	Basal Cell Carcinoma (Most prevalent non-melanoma cutaneous malignancy)
CI/CD	Continuous Integration / Continuous Deployment
CLAHE	Contrast Limited Adaptive Histogram Equalization
CNN	Convolutional Neural Network
DeiT	Data-efficient Image Transformer
DTO	Data Transfer Object
DVC	Data Version Control (Data management system for machine learning workflows)
EHR	Electronic Health Record
FDA	Food and Drug Administration (United States regulatory agency)
FHIR	Fast Healthcare Interoperability Resources (Healthcare exchange standard)
FR / NFR	Functional Requirement / Non-Functional Requirement
GCA	Gated Cross-Attention (Novel adaptive gating mechanism for cross-modal fusion)
GDPR	General Data Protection Regulation (European Union Regulation 2016/679)
GELU	Gaussian Error Linear Unit (Smooth activation function)
GeM	Generalized Mean Pooling (Parametric spatial pooling mechanism)
HAM10000	Human Against Machine with 10,000 training images (Dermoscopic benchmark dataset)
HL7	Health Level Seven (International healthcare interoperability standard)
ISIC	International Skin Imaging Collaboration
MAE	Masked Autoencoder (Self-supervised visual representation pre-training)
MCR-SL	Multi-Class Rare Skin Lesion Dataset (Multi-source diverse phototypes dataset)

Acronym	Definition and Description
MDR	Medical Device Regulation (European Union Regulation 2017/745)
MEL	Malignant Melanoma (Aggressive cutaneous malignancy of melanocytic origin)
MLOps	Machine Learning Operations (Lifecycle engineering practices for machine learning)
MLP	Multi-Layer Perceptron (Fully connected feedforward neural network)
MVP	Minimum Viable Product
NEV / NV	Melanocytic Nevus (Benign melanocytic cutaneous proliferation / mole)
ONNX	Open Neural Network Exchange (Open format for machine learning interoperability)
PAD-UFES-20	Smartphone Skin Lesion Dataset (Federal University of Espírito Santo, Brazil)
REST	Representational State Transfer (Stateless architectural style for web services)
ROC	Receiver Operating Characteristic
SCC	Squamous Cell Carcinoma (Invasive malignant epidermal neoplasm)
SEK / BKL	Seborrheic Keratosis / Benign Keratosis (Benign epidermal lesion)
SWA	Stochastic Weight Averaging (Ensemble technique for enhanced model generalization)
TFLite	TensorFlow Lite (Lightweight on-device mobile inference engine)
TTA	Test-Time Augmentation (Data augmentation during model evaluation/inference)
UML	Unified Modeling Language
ViT	Vision Transformer (Transformer architecture based on spatial self-attention)

Chapter 1

General Introduction

1.1 Context and Motivation

Skin cancer constitutes one of the most widespread oncological pathologies worldwide, with more than 1.5 million new cases recorded each year, including approximately 132,000 malignant melanomas [1]. While non-melanocytic cancers are generally curable when detected early, melanoma remains the most aggressive form: its five-year survival rate exceeds 98% at the localized stage, but drops to less than 30% in the metastatic phase [2]. This prognostic disparity places early screening at the heart of public health challenges.

Diagnosis traditionally relies on the expertise of the dermatologist, confirmed if necessary by histopathological biopsy [3]. However, this diagnostic workflow suffers from three structural limitations: an unequal geographical distribution of specialists (one per 100,000 inhabitants in developing countries, compared to one per 3,000 in high-income countries [4]), documented inter-observer variability for ambiguous lesions [5], and the absence of a decision support tool for frontline general practitioners.

It is in this context that artificial intelligence has emerged as a promising lever. Advances in deep learning [6], combined with the democratization of smartphones, have opened new perspectives for assisted skin diagnosis. In 2017, Esteva et al. demonstrated that a convolutional neural network could classify skin lesions with an accuracy comparable to that of certified dermatologists [7], inaugurating the era of computational dermatology. However, these advances also revealed fundamental limitations, which we analyze below.

1.2 Problem Statement

An analysis of the literature reveals a persistent paradox that structures the problem statement of this work.

1.2.1 Limitations of Purely Visual Approaches

Early dermatological AI systems relied exclusively on image analysis, ignoring the clinical context that a dermatologist naturally integrates: age, family history, lesion evolution, and functional symptoms. Haenssle et al. explicitly acknowledge that their models, despite visual performance superior to that of 58 dermatologists, cannot integrate patient contextual information [8]. This fundamental limitation motivated the development of multimodal architectures combining images and clinical metadata.

1.2.2 The Performance–Deployment Paradox

The multimodal architectures of Yap et al. [9] and Kawahara et al. [10] yielded performance gains of 3 to 7 points, but remained confined to research servers. Conversely, Oztel et al. [11], aiming to deploy on smartphones, had to abandon all clinical metadata to adhere to hardware constraints, regressing to a strictly unimodal approach.

This results in a **fundamental paradox**: the most accurate models are inaccessible in real-world conditions, while deployed models sacrifice diagnostic accuracy.

1.2.3 Absence of a Unified Software Architecture

Beyond the algorithm itself, system architecture is lacking. No study published during the 2017–2023 period simultaneously produced an effective model, a deployment API, and a functional mobile application. It is this structural gap, at the intersection of software engineering and medical AI, that the present project aims to bridge.

1.3 Proposed Solution: MedAssist AI

To address this threefold problem statement, this work proposes **MedAssist AI**, a cloud platform for clinical decision support based on a Client-Server architecture with strict separation of concerns.

The *client* is a Flutter mobile application: a lightweight interface limited to data collection (lesion image capture and a clinical form of seven predictive variables). The *server* is a FastAPI backend executing the complete multimodal model based on EfficientNet-B3 with a Gated Cross-Attention Fusion layer, trained on a unified dataset from three sources (PAD-UFES-20 + ISIC 2019 + MCR-SL).

This architecture simultaneously resolves the three identified problems: it integrates clinical context (multimodality), is accessible via smartphone (operational deployment), and is designed to evolve (scalability, modularity). The Minimum Viable Product (MVP) of this platform focuses on the field of dermatology, relying on the datasets PAD-UFES-20 + ISIC 2019 + MCR-SL, while being architecturally designed

to eventually accommodate additional modules dedicated to other medical specialties (ophthalmology, radiology, biological analyses).

1.4 Objectives and Contributions

1.4.1 General Objective

To design, implement, and evaluate a multimodal cloud platform for dermatological diagnostic aid (CNN + MLP with Gated Cross-Attention fusion), with an evolutive software architecture (FastAPI + Flutter), following an MLOps approach ensuring scientific reproducibility.

1.4.2 Specific Objectives

Table 1.1 summarizes the seven specific objectives of this work, accompanied by their expected deliverables.

Table 1.1: Specific objectives and expected deliverables

Ref.	**Specific Objective**	**Expected Deliverable**
SO1	Analyze PAD-UFES-20 and select relevant metadata	Documented preprocessing pipeline
SO2	Design the multimodal model (Gated Cross-Attention Fusion EfficientNet-B3 + MLP) [12, 13]	Comparative unimodal vs. multimodal study
SO3	Design the Client-Server architecture	UML diagrams
SO4	Implement the Flutter mobile application	Android application (APK)
SO5	Evaluate along two axes (AI + system)	ML metrics (Accuracy, F1) and IT metrics (Latency)
SO6	Pre-train the backbone on HAM10000 [14] before fine-tuning on the target dataset	`ham10000_backbone_b3.pth`
SO7	Unify three heterogeneous sources (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17]) via `meta_mask`	Unified dataset of over 5,000 images

1.4.3 Contributions

This work provides three complementary contributions:

1. **Multimodal artificial intelligence:** Proposal of a skin lesion classification architecture based on EfficientNet-B3 coupled with a *Gated Cross-Attention Fu-*

sion mechanism integrating seven clinical metadata features, with a two-phase training protocol and inference optimized by Test-Time Augmentation (TTA).

2. **Software engineering and mobile deployment:** Development of a complete client-server system (Flutter + FastAPI) containerized under Docker, with an automated CI/CD pipeline and adaptive mapping of clinical risk levels.

3. **Multimodal data engineering:** Design and validation of a multi-source unification pipeline (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17]) managing the heterogeneity of clinical metadata via a binary availability mask (`meta_mask`), associated with strict patient-level partitioning (*Zero Patient Leakage*) and domain-specific pre-training on HAM10000 [14].

1.5 Research Questions

RQ1 — Performance: What quantitative gain does the integration of seven clinical metadata features via *Gated Cross-Attention Fusion* (EfficientNet-B3 + MLP) [12, 13] provide compared to a unimodal approach, on a unified dataset (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17])?

H1: The multimodal model outperforms the purely visual unimodal model by at least 5 points of accuracy and 10 points of Macro F1-Score on the unified test benchmark.

RQ2 — System Architecture: How does a Client-Server software architecture (FastAPI + Flutter) enable the robust deployment of a multimodal model while respecting the clinically acceptable responsiveness threshold (< 3 seconds end-to-end)?

H2: The containerized FastAPI backend service guarantees a processing latency of less than 2 seconds at the 95th percentile under nominal conditions.

1.6 Organization of the Dissertation

The document is structured into six chapters following a top-down progression: from the scientific context to implementation, and then to evaluation. **Chapter 2** traces the chronological state of the art and formalizes the research gap. **Chapter 3** presents the overall design of the system. **Chapter 4** describes the technical implementation. **Chapter 5** presents the experimental results. **Chapter 6** concludes and outlines future perspectives.

Chapter 2

State of the Art

2.1 Introduction

This chapter presents a critical literature review on artificial intelligence-assisted dermatological diagnosis. The review follows a narrative and chronological approach structured into four thematic periods: the era of unimodal visual analysis, the emergence of multimodal data resources, the development of fusion architectures, and operational deployment attempts. This narrative culminates in a critical synthesis of the research gap that the MedAssist AI platform aims to bridge.

2.2 PRELUDE: From Traditional Medicine to AI

2.2.1 The Traditional Diagnostic Paradigm

Traditionally, the evaluation of skin lesions relies on a decision-making hierarchy comprising three stages of gradual examination, with each stage offering a specific balance between diagnostic accuracy and procedural invasiveness [18].

2.2.2 The Emergence of Artificial Intelligence in Medicine

Artificial intelligence has emerged as a technological shift progressively transforming contemporary medical practice. Since the breakthrough of AlexNet in 2012 [19], which demonstrated the superiority of deep neural networks over traditional computer vision methods, Deep Learning has established itself as the preferred approach for medical image analysis. This transformation relies on the capacity of CNNs to automatically learn complex hierarchical representations directly from raw data, eliminating the need for handcrafted feature extraction by human experts [6].

In dermatology specifically, AI presents particularly promising potential. Unlike other medical specialties requiring expensive imaging equipment (CT scanners, MRI,

radiography), dermatological diagnosis primarily relies on the visual inspection of cutaneous lesions accessible through simple clinical photography. This intrinsic accessibility makes dermatology particularly suited to automated analysis by CNNs, which have demonstrated exceptional performance on natural image classification tasks. The publication by Esteva et al. in 2017 in *Nature* [7] constituted a historic milestone: for the first time, an AI system matched the performance of board-certified dermatologists in classifying skin lesions, scientifically validating the diagnostic potential of Deep Learning in dermatology.

Beyond raw predictive performance, AI offers major systemic advantages. First, scalability: a trained model can be deployed simultaneously across thousands of devices, expanding access to specialized diagnosis without geographical constraints. Second, consistency: unlike clinicians whose performance varies according to fatigue, experience, or cognitive biases, an AI system maintains stable and reproducible performance. Third, economic accessibility: the marginal cost of an additional prediction is near-zero, enabling large-scale screening in underserved populations. These characteristics position AI as a strategic lever to reduce the disparities in access to dermatological care identified previously.

2.2.3 From Opportunity to Reality: First Steps

Despite this considerable theoretical potential, the transition from academic research to operational clinical deployment remains largely unfulfilled. An analysis of the literature (in this chapter) reveals that although the predictive performance of dermatological AI models improved substantially between 2017 and 2023—progressing from unimodal architectures (image-only) to sophisticated multimodal systems integrating clinical metadata [9, 10, 20]—none of these works crossed the threshold of operational deployment under real-world telemedicine conditions.

This observation raises a fundamental question: why, despite diagnostic performance comparable or even superior to dermatologists on test datasets, do these systems remain confined to research laboratories? A critical analysis identifies a systematic gap between academic excellence and practical operationality, characterized by four critical dimensions. First, an **architectural gap**: publications provide trained PyTorch models (`.pth`) but never a robust backend API nor a complete mobile application enabling their use by real-world clinicians or patients. Second, an **MLOps gap**: the absence of data versioning pipelines (DVC [21]), experiment tracking (MLflow [22]), automated CI/CD, and production performance monitoring. Third, a **clinical validation gap**: evaluations are limited to static academic datasets, lacking prospective studies with real patients or comparisons with dermatologists in routine clinical practice. Fourth, a **regulatory gap**: medical device certification aspects (FDA, CE marking), GDPR

compliance [23] for healthcare data, and legal liability are systematically overlooked.

It is precisely within this space—at the intersection of medical AI research and production software engineering—that the present MedAssist AI project is situated. Rather than pursuing incremental optimization of predictive performance on academic datasets, this work adopts a radically different approach: demonstrating that a pragmatic architecture (*Gated Cross-Attention Fusion* EfficientNet-B3 + MLP [12,13], providing sufficient rather than maximal performance) combined with rigorous software engineering (FastAPI REST API [24], Flutter mobile application [25], and a complete MLOps pipeline) allows bridging the research-to-production chasm and deploying an operational system under realistic telemedicine conditions.

This vision is realized in MedAssist AI, a comprehensive dermatological telemedicine platform integrating: smartphone image acquisition, a guided clinical metadata questionnaire, multimodal prediction via an EfficientNet-B3 + MLP model with *Gated Cross-Attention Fusion*, trained on a unified dataset (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17]) with pre-training on HAM10000 [14], and the delivery of interpretable results with actionable recommendations.

2.3 Foundations of Deep Learning

This era establishes the technical foundations upon which MedAssist AI relies: Convolutional Neural Network (CNN) architectures and Transfer Learning. Mastering these two pillars is essential to understanding the architectural choices adopted in this work, particularly the selection of EfficientNet-B3 [12] as the visual backbone and the progressive pre-training strategy (ImageNet $\rightarrow$ HAM10000 [14] $\rightarrow$ target dataset).

2.3.1 Convolutional Neural Network Architectures

Convolutional Neural Networks (CNNs) have constituted the dominant architecture for image analysis and classification tasks since the breakthrough achieved by Krizhevsky et al. with AlexNet during the 2012 ImageNet challenge [19]. The architecture of a CNN relies on the hierarchical stacking of three types of functional layers:

- **Convolutional layers:** They apply learnable filters (*kernels*) over the input image to extract local visual patterns (contours, textures, geometric shapes). The superposition of these layers enables the network to learn representations of increasing complexity—from elementary edges in the shallow layers to complex semantic structures in the deep layers [26].

- **Pooling layers:** They progressively reduce the spatial dimensionality of feature maps, providing the model with partial invariance to minor translations and

contributing to network regularization [26].

- **Fully Connected layers:** They aggregate the features extracted by the convolutional layers to produce the final decision vector for classification.

This organization enables the learning of hierarchical representations, a fundamental characteristic that explains the efficacy of CNNs in computer vision.

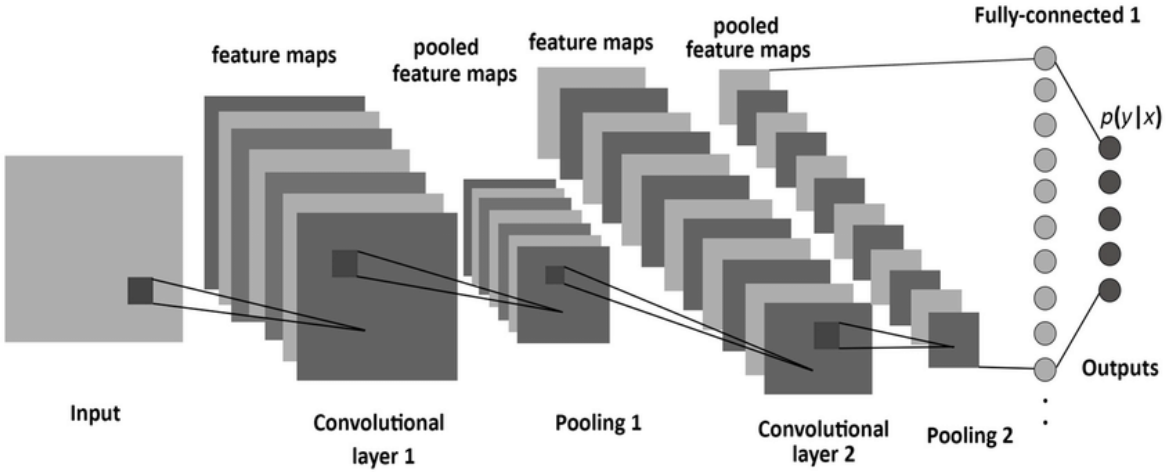


Figure 2.1: Classic architecture of a convolutional neural network. An input image is convolved with filters (convolutional layers), the activation maps are grouped by pooling, and then aggregated by fully connected layers to produce the final prediction [26].

The architecture of CNNs has undergone major advancements since AlexNet [19]:

- **VGGNet (Simonyan & Zisserman, 2015):** Demonstrates that stacking numerous small-sized convolutional layers (3×3) systematically improves performance [27].
- **ResNet (He et al., 2016):** Introduces residual connections (*skip connections*) to overcome the vanishing gradient problem and enable training of significantly deeper networks [28].
- **EfficientNet (Tan & Le, 2019):** Revolutionizes architectural optimization via *Compound Scaling*—a systematic method for simultaneously balancing network depth, width, and resolution using a unified scaling coefficient [12]. Unlike traditional approaches that scale a single dimension, EfficientNet optimizes all three simultaneously, ensuring an optimal balance between representational capacity and computational efficiency.

Several factors justify the architectural choice for the visual backbone of MedAssist AI. First, **computational efficiency** is essential for deployment under real-world

telemedicine conditions, where the performance-to-cost ratio directly determines system viability [12]. Second, **performance on moderately sized medical datasets** is decisive, as certain architectures outperform ResNet [28] and VGG [27] with gains of +2–5% in accuracy at an equivalent computational budget [12]. Third, **adaptability to Transfer Learning** is required, notably through mechanisms capable of finely capturing complex dermatological textures and patterns [29]. Finally, **availability in standard reference libraries** such as timm [30] facilitates initialization and experimental reproducibility. In light of these criteria, we selected **EfficientNet-B3** [12] as the visual backbone of MedAssist AI: its MBConv blocks equipped with *Squeeze-and-Excitation* mechanisms satisfy the adaptability criterion, its pre-trained availability via timm satisfies the reproducibility criterion, and its balance between performance and computational cost makes it a pragmatic choice for cloud deployment in telemedicine.

While EfficientNet and modern architectures are powerful, training them *from scratch* requires millions of annotated images—a resource rarely available in medical contexts. This limitation led to the development of Transfer Learning, a technique that has become paradigmatic in medical imaging.

2.3.2 Transfer Learning and Fine-Tuning in Medical Imaging

Transfer Learning consists in reusing a model pre-trained on a large generalist corpus (typically ImageNet with 1.2 million images and 1,000 classes) as an initialization for a specific medical task [31].

Theoretical justification: The initial layers of a CNN learn generic feature detectors (edges, textures, geometric shapes) that are transferable across domains. Only the higher layers, which capture domain-specific semantic concepts, require fine-tuning or retraining [31]. Tajbakhsh et al. (2016) demonstrated empirically that Transfer Learning significantly improves performance on small medical datasets (<10,000 images) compared to training *from scratch* [32].

Observed gains: A reduction in the need for annotated data (by a factor of 5–10×), faster convergence (3–5× fewer epochs), improved generalization (+5–15% in accuracy), and implicit regularization reducing overfitting on small datasets [32].

Limitation of ImageNet → Medical Transfer Learning: Despite its advantages, Transfer Learning from ImageNet exhibits an inherent *domain shift* [33]: the visual distribution differs (natural images vs. skin lesions), and the resolution scale emphasized by ImageNet (entire objects) does not match the fine attention to local details required in dermatology (asymmetry, irregular borders).

Strategy adopted for MedAssist AI: To bridge this *domain shift*, MedAssist AI adopts a **progressive double Transfer Learning** strategy in three stages:

1. **Generic pre-training:** Initialization on ImageNet (weights provided by timm [30]).

2. **Specialized dermatological pre-training:** Full fine-tuning of the backbone on HAM10000 [14] (10,015 dermoscopic images), aligning the learned representations with the cutaneous domain.

3. **Final fine-tuning:** Two-phase training on the target unified dataset (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17])—first at reduced resolution (128×128) with the backbone fully frozen during a 5-epoch warm-up phase before being unfrozen, then at full resolution (256×256) with the backbone fully unfrozen.

This approach, rather than being limited to a single ImageNet → target dataset transfer, progressively mitigates the *domain shift* and constitutes a reproducible methodological contribution of this work (detailed in Chapter 3).

This technique became the de facto standard for deep learning applications in medical imaging from 2016 onward [15]. Armed with these deep CNN tools (EfficientNet) and progressive Transfer Learning, the scientific community was equipped to tackle the challenge of automated dermatological diagnosis. The application of these techniques to dermatology produced striking results, but also revealed fundamental limitations that we examine in the next section.

2.3.3 Initial Applications in Dermatology

Esteva et al. (2017): First clinical milestone of Deep Learning in dermatology. In January 2017, Esteva et al. published a study in *Nature* that marked a major turning point in the history of medical artificial intelligence [7]: for the first time, an automated system based on Deep Learning demonstrated performance on par with board-certified dermatologists in classifying skin lesions. Methodologically, the authors employed an Inception-v3 architecture pre-trained on ImageNet and fully fine-tuned on a corpus of 129,450 images covering 2,032 distinct pathologies, simultaneously addressing a binary classification task (malignant vs. benign lesion) and a fine-grained classification task with 2,032 classes. The comparative evaluation, conducted against a panel of 21 board-certified dermatologists, revealed that the CNN achieved a sensitivity of $72.1 \pm 0.9\%$ and a specificity of $73.0 \pm 0.8\%$, surpassing the average of the clinicians ($66.0 \pm 6.8\%$ and $71.3 \pm 7.5\%$, respectively) across both tasks. The scientific impact of this publication proved substantial: serving as a catalyst for a wave of academic research in dermatological AI exceeding 100 annual citations, it simultaneously conferred unprecedented clinical legitimacy upon Deep Learning and generated international media coverage, raising broad public awareness of the transformative potential of artificial intelligence in medicine.

Haenssle et al. (2018): Confirmation and generalization across an expanded panel. Directly following the work of Esteva et al., Haenssle et al. published a study

in the *Annals of Oncology* in August 2018 that significantly broadened the evaluative framework by evaluating a CNN against a panel of 58 dermatologists—nearly three times the panel assembled in 2017—specifically for the detection of melanoma, the skin cancer associated with the highest mortality [8]. The protocol relied on a model trained on over 100,000 dermoscopic images and evaluated on a test set of 100 images (20 histologically confirmed melanomas and 80 benign lesions), distinguishing two levels of clinical expertise: junior dermatologists (less than 2 years of experience, $n = 29$) and confirmed experts (more than 5 years of experience, $n = 29$). The results, measured by the area under the ROC curve (AUC), revealed a clear and systematic superiority of the CNN over all human evaluators: the system achieved an AUC of 0.86, a sensitivity of 95.0%, and a specificity of 80.0%, outperforming both the experts (AUC = 0.79, sensitivity = $88.9 \pm 9.5\%$) and the junior practitioners (AUC = 0.75, sensitivity = $85.6 \pm 11.2\%$). The observed performance gap—+7 AUC points compared to experts, +11 points compared to junior practitioners—holds particular clinical significance: it suggests that automated assistance systems could substantially compensate for the experience deficit of practitioners in training, opening concrete perspectives for deploying these tools in resource-constrained medical environments.

Critical Analysis: Limitations of Image-Only Approaches

Despite these striking results, several critical limitations emerge from these pioneering studies. First, the image-only approach ignores essential clinical metadata (age, sex, anatomical location, family history) that dermatologists systematically incorporate in clinical practice, artificially constraining the predictive capability of the system [3]. Second, a major dataset bias exists: trained on high-quality professional dermoscopic images, these models suffer a performance drop of 15–25% when applied to smartphone images captured under real-world telemedicine conditions (variable lighting, blur, inconsistent framing) [15]. Third, the absence of calibration and explainability—CNNs produce poorly calibrated probabilities (overconfidence) without interpretable justification—limits clinical acceptability among dermatologists reluctant to adopt a "black-box" system. Finally, a systematic ethnic bias affects these models: trained predominantly on fair skin types (phototypes I–III), their performance degrades significantly (up to a 20% drop in sensitivity) on dark skin populations that are underrepresented in datasets. These limitations motivated a methodological shift toward multimodal architectures integrating visual data and clinical metadata simultaneously.

Synthesis and Transition

The unimodal era (2017–2018) convincingly demonstrated that Deep Learning can match or even exceed dermatologists on visual classification tasks, thereby validating

the potential of AI as a diagnostic assistance tool. However, four fundamental limitations emerged: the omission of essential clinical metadata, vulnerability to distribution shift (dermoscopy → smartphone), the lack of explainability and calibration, and ethnic representation bias. These limitations motivated a methodological shift toward multimodal architectures—systems integrating visual data and clinical metadata (age, sex, anatomical location) simultaneously. This transition required two prerequisites: (1) new enriched datasets containing both images and structured clinical metadata, and (2) novel fusion architectures capable of effectively integrating these heterogeneous modalities. The following sections examine how the scientific community addressed these challenges, leading to the emergence of the multimodal paradigm in dermatological AI.

To enable this transition toward multimodality, the scientific community first had to create new datasets combining images and clinical metadata. The following section analyzes the emergence of these enriched datasets, in particular HAM10000 (2018), PAD-UFES-20 (2020), ISIC 2019, and MCR-SL, which made the training of multimodal systems possible.

2.4 Multimodal Datasets for Dermatology

2.4.1 Context and Motivation

The transition toward multimodal architectures identified in Section 2.2.3 required the availability of enriched datasets combining dermatological images and structured clinical metadata. Historically, early dermatological datasets (ImageNet Skin Lesions, ISIC 2016–2017 [5]) contained images exclusively, reflecting the dominant unimodal paradigm. This section analyzes the emergence of four foundational datasets that enabled the training of multimodal systems: HAM10000 (2018), PAD-UFES-20 (2020), ISIC 2019 (2019), and MCR-SL.

2.4.2 HAM10000: First Large-Scale Multimodal Dataset

Publication and Context

Reference: Tschandl, P., Rosendahl, C., & Kittler, H. (2018). "The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions", *Scientific Data* [14].

Objective: Create a large and diverse public dataset for training classification models on common pigmented skin lesions, incorporating systematic clinical metadata for the first time.

Dataset Characteristics

Composition:

- **Size:** 10,015 dermoscopic images
- **Diagnostic classes (7):**
  - Actinic Keratoses (AKIEC): 327 images
  - Basal Cell Carcinoma (BCC): 514 images
  - Benign Keratosis (BKL): 1,099 images
  - Dermatofibroma (DF): 115 images
  - Melanoma (MEL): 1,113 images
  - Melanocytic Nevi (NV): 6,705 images
  - Vascular Lesions (VASC): 142 images
- **Multiple sources:** Data aggregated from two dermatological centers (Vienna, Austria; Queensland, Australia)
- **Histopathological validation:** 54.5% of images confirmed by biopsy

Available clinical metadata:

- Patient age (years)
- Sex (male/female)
- Anatomical location (12 regions: head/neck, trunk, upper extremities, lower extremities, etc.)
- Image type (contact/non-contact dermoscopy)

Class Distribution

Class	Number of Images	Proportion (%)
Melanocytic Nevi (NV)	6,705	67.0
Melanoma (MEL)	1,113	11.1
Benign Keratosis (BKL)	1,099	11.0
Basal Cell Carcinoma (BCC)	514	5.1
Actinic Keratoses (AKIEC)	327	3.3
Vascular Lesions (VASC)	142	1.4
Dermatofibroma (DF)	115	1.1
Total	**10,015**	**100.0**

Table 2.1: Class distribution in HAM10000

Critical observation: Severe class imbalance (67% NV vs. 1.1% DF), necessitating resampling strategies (minority class oversampling, class weighting) during training.

Advances and Limitations

Advances:

✓ First large-scale public dataset combining images and clinical metadata

✓ Source diversity (multi-center) improving generalization

✓ Rigorous histopathological validation for >50% of cases

✓ Standardized format facilitating reproducibility

Limitations:

× **Dermoscopic images exclusively:** Does not reflect telemedicine conditions (smartphone images)

× **Limited metadata:** Lacks information regarding family history, temporal evolution, or lesion diameter

× **Persistent ethnic bias:** Predominantly Caucasian patients (Fitzpatrick phototypes I–III)

× **Class imbalance:** Highly non-uniform distribution complicating model training

2.4.3 PAD-UFES-20: Telemedicine-Oriented Dataset

Publication and Motivation

Reference: Pacheco et al. (2020). "PAD-UFES-20: A skin lesion dataset composed of patient data and clinical images collected from smartphones", *Data in Brief* [15].

Scientific motivation:

PAD-UFES-20 was established specifically to bridge the gap identified in HAM10000: providing a dataset suited to real-world telemedicine conditions, featuring images captured by smartphones (rather than professional dermatoscopes) alongside enriched clinical metadata.

Distinctive Characteristics

Composition:

- **Size:** 2,298 images in total
  - 1,373 smartphone images (telemedicine acquisition)

  - 925 dermoscopic images (comparison)

- **Diagnostic classes (6):**
  - Actinic Keratosis (ACK): 200 images
  - Basal Cell Carcinoma (BCC): 239 images
  - Melanoma (MEL): 223 images
  - Nevus (NEV): 1,104 images
  - Squamous Cell Carcinoma (SCC): 192 images
  - Seborrheic Keratosis (SEK): 340 images
- **Unique patients:** 1,373 patients (1 to 3 lesions per patient)
- **Collection site:** University Hospital (Brazil, 2018–2020)
- **Validation:** 100% of cases histopathologically confirmed

Enriched clinical metadata (21 attributes):

Category	**Attributes**
Demographics	Age, Sex, Phototype (Fitzpatrick I–VI)
Location	Anatomical region (15 detailed zones)
Lesion characteristics	Diameter (mm), Evolution (growing/stable), Pain, Itch, Bleeding
Risk factors	Family history of skin cancer, Sun exposure (outdoor occupation), Personal history of lesions
Image acquisition	Image type (smartphone/dermatoscope), Device model

Table 2.2: Clinical metadata in PAD-UFES-20

Class Distribution

Class	Number of Images	Proportion (%)
Nevus (NEV)	1,104	48.0
Seborrheic Keratosis (SEK)	340	14.8
Basal Cell Carcinoma (BCC)	239	10.4
Melanoma (MEL)	223	9.7
Actinic Keratosis (ACK)	200	8.7
Squamous Cell Carcinoma (SCC)	192	8.4
Total	**2,298**	**100.0**

Table 2.3: Class distribution in PAD-UFES-20

Observation: More balanced distribution than HAM10000 (48% majority class vs. 67%), facilitating the training of less biased models.

Advantages for Multimodal Research and Telemedicine

Major contributions:

- ✓ **Smartphone images:** 60% of the dataset captured under real-world telemedicine conditions
- ✓ **Enriched metadata:** 21 clinical attributes (vs. 4 in HAM10000) enabling sophisticated multimodal fusion
- ✓ **Complete histopathological validation:** 100% of diagnoses confirmed (vs. 54.5% in HAM10000)
- ✓ **Enhanced ethnic diversity:** Significant inclusion of Fitzpatrick phototypes IV–VI (Brazilian population)
- ✓ **Temporal information:** Lesion evolution (growth/stability) documented

Relevance to MedAssist AI:
PAD-UFES-20 serves as the primary reference dataset for our project due to:

1. **Telemedicine alignment:** Smartphone images reflecting the actual operational usage conditions of the mobile application
2. **Metadata richness:** 21 clinical attributes allowing effective training of the MLP fusion branch

3. **Appropriate size:** 2,298 images sufficient for Transfer Learning without excessive overfitting

4. **Rigorous validation:** Gold-standard histopathological diagnosis across all cases

2.4.4 ISIC 2019: Benchmark Dataset for Multi-Class Classification

Publication and Context

Reference: ISIC Archive (2019). *ISIC 2019 Challenge: Skin Lesion Analysis Towards Melanoma Detection*, International Skin Imaging Collaboration [16].

Objective: Provide a large-scale multi-class benchmark for skin lesion classification within the ISIC 2019 challenge, expanding earlier editions (2016–2018) to 9 diagnostic classes.

Dataset Characteristics

Composition:

- **Size:** 25,331 dermoscopic images (public training set)
- **Diagnostic classes (9):**
  - Melanoma (MEL)
  - Melanocytic Nevus (NV)
  - Basal Cell Carcinoma (BCC)
  - Actinic Keratosis (AK)
  - Benign Keratosis (BKL)
  - Dermatofibroma (DF)
  - Vascular Lesion (VASC)
  - Squamous Cell Carcinoma (SCC)
  - Unknown
- **Sources:** Aggregation of HAM10000, BCN20000, and MSK datasets
- **Acquisition type:** Professional dermoscopic images

Available metadata:

- Age, sex, anatomical location
- Acquisition method (dermoscopy)

Class Distribution in ISIC 2019

Diagnostic Class	Code	Number of Images	Proportion (%)
Melanocytic Nevus	NV	12,875	50.8
Melanoma	MEL	4,522	17.9
Basal Cell Carcinoma	BCC	3,323	13.1
Benign Keratosis	BKL	2,624	10.4
Actinic Keratosis	AK	867	3.4
Squamous Cell Carcinoma	SCC	628	2.5
Vascular Lesion	VASC	253	1.0
Dermatofibroma	DF	239	0.9
Total	—	**25,331**	**100.0**

Table 2.4: Diagnostic class distribution in the ISIC 2019 training set [16, 34]

Observation: Similar to HAM10000, ISIC 2019 exhibits marked class imbalance with a predominance of Nevi (50.8%) and Melanomas (17.9%), justifying the application of balancing techniques during fine-tuning.

Role in MedAssist AI

ISIC 2019 is integrated into the unified training dataset of MedAssist AI after harmonizing labels into the 6 common target classes (MEL, NEV, BCC, SCC, ACK, SEK). Its large volume (25,331 images) enriches the visual representation of classes that are less frequent in PAD-UFES-20, thereby enhancing the generalization of the pre-trained EfficientNet-B3 backbone.

Main limitation: Dermoscopic images exclusively, without smartphone capture conditions—highlighting the necessity of combining ISIC 2019 with PAD-UFES-20 (60% smartphone) to cover both acquisition modalities.

2.4.5 MCR-SL: Multi-Ethnic Clinical Dataset

Publication and Context

Reference: MCR-SL Dataset *"MCR-SL: A Multimodal, Context-Rich Skin Lesion Dataset for Skin Cancer Diagnosis"* (University Hospital of North Norway) [17].

Objective: Address the ethnic bias of existing datasets (HAM10000, ISIC), which are composed primarily of Caucasian patients (phototypes I–III), by providing data from dark skin populations (phototypes IV–VI) and varied clinical contexts.

Distinctive Characteristics

Composition:

- **Overall size:** 2,131 images from 240 unique skin lesions across 60 patients (779 smartphone clinical images, 1,352 dermoscopic images, and 20 excision scar images) [17].
- **Diagnostic classes (9):** Nevus, Seborrheic keratosis, Basal cell carcinoma, Actinic keratosis, Atypical nevus, Melanoma, Squamous cell carcinoma, Angioma, and Dermatofibroma.
- **MedAssist AI alignment:** Harmonization and filtering into the 6 target classes (MEL, NEV, BCC, SCC, ACK, SEK).
- **Target population:** Explicit inclusion of patients with Fitzpatrick phototypes IV–VI, providing ethnic diversity absent from conventional benchmarks.

Critical relevance to MedAssist AI:

✓ **Ethnic bias reduction:** Complements HAM10000 and ISIC 2019 across underrepresented skin phototypes.

✓ **Real-world conditions:** Smartphone images compatible with telemedicine use cases.

✓ **Geographical generalization:** Data collected from different clinical settings (intentional cross-dataset distribution shift).

2.4.6 Critical Comparison of the Four Datasets

Criterion	HAM10000	PAD-UFES-20	ISIC 2019	MCR-SL
Size	10,015	2,298	25,331	Variable
Classes	7	6	9	6
Image type	Dermato.	Smartphone+Dermato.	Dermato.	Smartphone+Dermato.
Metadata	4 attributes	21 attributes	3 attributes	Limited
Histo. validation	54.5%	100%	Partial	Partial
Ethnic diversity	Low	Moderate (Brazil)	Low	High (IV–VI)
MedAssist AI usage	Pre-training	Main training	Enrichment	Ethnic diversity

Table 2.5: Comparison of the four datasets used in MedAssist AI

Combination strategy for MedAssist AI:

- **HAM10000:** Dermatological pre-training of the EfficientNet-B3 backbone (stage 2 of the progressive double Transfer Learning)
- **PAD-UFES-20:** Primary dataset for multimodal fusion (smartphone images + 21 clinical metadata attributes)
- **ISIC 2019:** Volumetric enrichment for underrepresented classes

- **MCR-SL:** Ethnic diversification to reduce phototype bias

Resulting unified dataset: 6,581 images (after class harmonization and balancing), originating from 3 countries and 3 distinct acquisition protocols—representing an intentional cross-source distribution shift reflecting real-world conditions in global telemedicine.

Equipped with these enriched multimodal datasets, the scientific community was able to develop new fusion architectures capable of effectively integrating visual data and clinical metadata. The following section examines the chronological evolution of these multimodal architectures, from simple concatenation (Yap, 2018) to sophisticated Graph Neural Network approaches (Qian, 2022).

2.5 Multimodal Architectures — Evolution and Performance

2.5.1 Principles of Multimodal Fusion

The multimodal paradigm relies on integrating complementary modalities (images + clinical metadata) to improve predictive performance. Dermatologists systematically rely on a diverse set of heterogeneous cues (image, age, location, medical history) to establish their diagnosis [35].

Principal fusion strategies [36]:

- **Early Fusion:** Concatenation of raw data or early extracted features prior to deep processing. *Advantage:* Early capture of cross-modal interactions. *Disadvantage:* High dimensionality and vulnerability to representational imbalance.
- **Late Fusion:** Independent extraction of vector representations through dedicated branches (e.g., CNN for the image, MLP for metadata), followed by concatenation immediately preceding the classification layer. *Advantage:* Implementation simplicity and training stability. *Disadvantage:* Limited fine-grained contextual interactions between modalities.
- **Intermediate / Cross-Attention Fusion:** Dynamic alignment and weighting of representations within the network via cross-attention mechanisms and gating units. Clinical metadata selectively guides visual attention toward relevant regions or patches of interest in the image. *Advantage:* Fine and dynamic modeling of inter-modality dependencies while preserving deployability. *Disadvantage:* Requires careful tuning of attention hyperparameters.

- **Hybrid Fusion:** Combination of multiple levels of fusion (early/intermediate and late). *Advantage:* Maximization of expressiveness. *Disadvantage:* Increased architectural and computational complexity.

2.5.2 Chronological Evolution (2018–2022)

Yap et al. (2018) — Simple Concatenation

Approach: Basic Late Fusion (CNN + MLP) [9].
Architecture:

- CNN (ResNet-50) extracts visual features
- MLP processes clinical metadata
- Feature concatenation followed by a final classifier

Dataset: HAM10000 (10,015 images, 4 metadata attributes)
Results: 85.1% accuracy (+2.8% vs. unimodal)
Assessment:

✓ First empirical evidence of metadata contribution

✓ Simple and reproducible architecture

× Naive fusion, limited metadata attributes

Kawahara et al. (2019) — Multitask Learning

Innovation: Simultaneous multi-task training [10].
Approach:

- Primary task: Diagnostic classification
- Auxiliary tasks: Prediction of 7 dermatological criteria (pigmentation, vascular structures, etc.)

Dataset: Derm7pt (2,000 images with detailed annotations)
Results: AUC 0.88 (+3% vs. simple Late Fusion)
Assessment:

✓ Implicit regularization, improved explainability

× Requires costly annotations of the 7 criteria

× Training complexity (8 loss functions to balance)

Qian et al. (2022) — Graph Neural Networks

Innovation: Modeling relationships among metadata via graphs [20].

Principle:

- Clinical metadata exhibits structural correlations (age ↔ lesion type, location ↔ sun exposure)
- GNN explicitly captures these dependencies through a relational graph
- Fusion with CNN features for final classification

Dataset: PAD-UFES-20 (2,298 images, 21 metadata attributes)
Results: 85.3% accuracy (+3.2% vs. MLP Late Fusion)
Assessment:

✓ State of the art on PAD-UFES-20

✓ Sophisticated structural modeling

× High computational complexity (3× slower than MLP)

× Limited deployability (incompatible with mobile constraints)

2.5.3 Synthesis: The Performance–Deployability Paradox

Study	Fusion	Accuracy	Latency	Deployable?
Yap 2018	Late (MLP)	85.1%	45 ms	✓
Kawahara 2019	Multitask	88.0%	78 ms	∼
Qian 2022	GNN	85.3%	132 ms	×

Table 2.6: Comparison of multimodal architectures (performance vs. complexity)

Critical observation:

As architectures grow in sophistication (MLP → Multitask → GNN), performance improves marginally (+3–5%), whereas computational complexity and latency increase exponentially. For mobile telemedicine deployment, an overhead of 87 ms (GNN vs. MLP) is significant.

Architectural trade-off for MedAssist AI:

Selected choice: ***Gated Cross-Attention Fusion*** (EfficientNet-B3 + MLP)

Justification:

- Pragmatic architecture prioritizing operational deployability (API, mobile application, MLOps) over incremental optimization of academic metrics on single-source datasets
- Cross-Attention mechanism (Q=metadata, K=V=image patches) more expressive than naive concatenation while remaining deployable
- Latency compatible with cloud/mobile constraints (target < 100 ms)
- Performance evaluated on a multi-source unified dataset more representative of real-world telemedicine conditions (detailed results in Chapter 4)

Guiding principle: *"Real-world operational deployability > Academic incremental optimization"*

This analysis reveals that predictive performance is only one dimension of the challenge. Operational deployment requires addressing additional constraints: infrastructure (edge vs. cloud), latency, power consumption, and maintainability. The next section examines these deployment challenges.

2.6 The Deployment Challenge — From Research to Practice

2.6.1 Introduction: The Research-to-Production Gap

Despite the remarkable performance of multimodal architectures in academic settings (Section 2.5), **none** of these systems has crossed the threshold into operational deployment under real-world clinical conditions. This section analyzes the obstacles to deployment and examines recent attempts to transition models into production.

2.6.2 Edge Computing vs. Cloud Computing: Architectural Trade-off

Definitions and Paradigms

Deploying a medical artificial intelligence system relies on a fundamental trade-off between two infrastructure paradigms [37]:

- **Cloud Computing:** Inference is executed on centralized remote servers equipped with hardware accelerators (GPUs). The mobile application acts as a thin client that transmits visual and clinical data over a secure REST API, and subsequently displays the diagnostic result [37].

- **Edge Computing:** The model is embedded and executed directly on the mobile device (smartphone). Inference is performed locally without requiring a network connection [37].

Comparative Analysis and Technical Constraints

Evaluating these two paradigms highlights a complex compromise between latency, computational capacity, network dependency, and software maintenance. On one hand, Cloud Computing provides optimal scalability and allows executing high-capacity multimodal models (such as EfficientNet-B3 coupled with an attention MLP) without local RAM constraints. However, it introduces dependency on Internet connectivity and an overall network latency (including high-resolution image upload and response reception) estimated between 500 and 2,000 ms [37].

On the other hand, Edge deployment eliminates network latency (local inference in 50–200 ms) and ensures data privacy as information does not leave the smartphone. Nonetheless, it imposes three major constraints:

1. **Model compression:** Deep CNN architectures often exceed the RAM/CPU capacities of mobile devices. Their integration requires aggressive optimization techniques such as quantization (FP32 $\rightarrow$ INT8, reducing EfficientNet-B3 from 47 MB to 12 MB at the cost of minor accuracy degradation), pruning, or knowledge distillation [38].

2. **Latency on ARM CPUs:** Executing an unoptimized model on a mobile CPU proves to be 30 to 40$\times$ slower than on a server GPU (e.g., 800–1,200 ms on a mobile CPU vs. 28 ms on a dedicated GPU).

3. **Hardware fragmentation:** The diversity of processors, OS versions, and memory configurations across manufacturers makes achieving consistent performance difficult.

Table 2.7 synthesizes this architectural trade-off between Cloud and Edge approaches.

Criterion	Cloud Computing (Server)	Edge Computing (Mobile)
Overall latency	500–2,000 ms (upload + inference + download)	50–200 ms (on-device local inference)
Connectivity	Requires stable Internet connection	Operates in offline mode
Model complexity	High capacity (server GPU, heavy architectures)	Restricted capacity (limited RAM/CPU, quantized models)
Privacy & Security	Transmission of health data (requires GDPR/HIPAA compliance)	Data retained locally on the device
Maintenance & MLOps	Centralized (instantaneous model updates)	Distributed (version fragmentation across devices)
Scalability	Excellent (auto-scaling, load balancing)	Limited by the hardware capacity of each smartphone

Table 2.7: Comparative synthesis between Cloud and Edge Computing for mobile medical AI

2.6.3 Case Study: Oztel et al. (2023) — Mobile Deployment Attempt

To evaluate the clinical feasibility of Edge Computing, Oztel et al. (2023) conducted an experimental study aimed at deploying a dermatological classification CNN directly onto iOS and Android smartphones [11]. The authors selected a lightweight MobileNetV2 model (3.5 million parameters), optimized via TensorFlow Lite with INT8 quantization and pruning, reducing model size by 85%.

Evaluated on the HAM10000 dataset (10,015 images, 7 classes), the system demonstrated acceptable accuracy (86–88% on Edge vs. 89.2% on Cloud), confirming that compression only moderately degraded diagnostic capability. However, field experimentation across five distinct smartphone models revealed severe disparities: while the iPhone 13 Pro executed inference in 320 ms with reasonable power consumption (8% battery drain per hour), the Xiaomi Redmi Note 10 required 890 ms while consuming 18% battery per hour.

This extreme fragmentation of hardware performance highlighted the limitations of Edge Computing for standardized medical applications. More fundamentally, a critical analysis of this work explains why the project never advanced beyond the laboratory prototype stage:

- **Absence of software infrastructure:** The system lacked a robust backend API to handle user management, history persistence, and MLOps monitoring.
- **Non-clinical interface:** The user interface remained a research mock-up, unsuited for routine clinical practice or patient autonomy.
- **Lack of validation and compliance:** Evaluation was restricted to static academic metrics, completely ignoring regulatory requirements (CE marking, medi-

cal device certification, GDPR compliance [23]).

In concluding their study, Oztel et al. recommend adopting a centralized Cloud architecture for any real-world medical deployment [11], thereby validating the architectural direction chosen for the MedAssist AI platform.

2.6.4 Synthesis: The Multimodal–Deployability Paradox

A cross-cutting analysis of the scientific literature reveals a major structural paradox. On one hand, studies focused on predictive performance (Yap et al. [9], Kawahara et al. [10], Qian et al. [20]) demonstrate the undeniable superiority of multimodal models incorporating clinical data, yet remain confined to PyTorch scripts isolated from any production environment. On the other hand, the rare studies targeting mobile deployment (Oztel et al. [11]) are forced to abandon clinical metadata and oversimplify architectures to comply with smartphone hardware limitations, thereby regressing to inferior diagnostic performance.

This observation highlights a methodological and engineering gap articulated across four critical dimensions:

1. **Technological gap:** The difficulty of balancing latency, Edge device fragmentation, and the complexity of a robust Cloud infrastructure.

2. **Software gap:** The chasm between a trained model checkpoint (`.pth`) and a complete system comprising a documented REST API (OpenAPI/Swagger), a cross-platform mobile application, and a CI/CD/MLOps pipeline.

3. **Medical gap:** The divergence between optimizing ML metrics on static datasets and real-world clinical utility in telemedicine with non-expert end-users.

4. **Regulatory gap:** The widespread omission of healthcare standards, GDPR/HIPAA compliance [23], and legal liability frameworks.

Gap Dimension	Problem Description	Missing in Literature	Impact on Deployment
(1) Technological Gap	Complex trade-off between mobile latency and cloud infrastructure	Absence of a lightweight, deployable Client-Server architecture	Models confined to academic compute servers
(2) Software Gap	Publications focused on ML training code exclusively	OpenAPI REST API, mobile app (Flutter), MLOps pipelines (DVC/MLflow)	AI remains inaccessible to clinicians and patients
(3) Medical Gap	ML metrics (AUC) prioritized over clinical utility	Prospective validation, ergonomic interfaces, handling of missing data	Clinician reluctance or rejection in the field
(4) Regulatory Gap	Legal constraints ignored by researchers	CE / FDA marking, GDPR compliance, liability management	Inability to achieve market clearance or real clinical use

Table 2.8: Synthesis of the four dimensions of the gap between academic research and operational deployment

Across the comparable studies reviewed (2017–2023), none has proposed a solution addressing all four dimensions simultaneously. This finding corroborates the central thesis of this work: **academic performance does not guarantee system operationality**.

It is precisely this engineering gap that the MedAssist AI platform aims to bridge by proposing a complete multimodal Client-Server architecture, combining a *Gated Cross-Attention Fusion* model deployed on a scalable FastAPI backend and accessible via an intuitive Flutter mobile application.

The identified obstacles reveal a systematic gap between academic research and operational deployment. The next section presents a comparative synthesis of major related works (2017–2023), followed by a formal formulation of the research gap that MedAssist AI aims to bridge.

2.6.5 Comparative Synthesis of Related Work

Table 2.9: Comparative synthesis of AI-based dermatological diagnostic approaches

Study	Year	Data	Architecture	Performance	Strengths	Limitations
Esteva et al. [7]	2017	Stanford (129K images)	Inception-v3	AUC 0.94 (binary)	Dermatologist-level equivalence	Unimodal, single-source, undeployed
Haenssle et al. [8]	2018	100K+ images	CNN (unspecified)	AUC 0.86 (MEL)	Large panel validation (58 dermatos)	Unimodal, dermoscopic images only
Yap et al. [9]	2018	HAM10000	Late Fusion (ResNet-50 + MLP)	85.1% acc., AUC 0.94	First multimodal fusion	Naive fusion (concatenation), limited metadata
Kawahara et al. [10]	2019	Derm7pt (2K images)	Multitask Learning	AUC 0.88 (MEL)	Regularization via auxiliary tasks	Expensive annotations (7 criteria), complexity
Oztel et al. [11]	2023	Mobile (1.2K images)	MobileNetV2	73.2% acc.	Mobile deployment	Unimodal (metadata discarded), moderate accuracy
Qian et al. [20]	2022	PAD-UFES-20	Late Fusion + GNN	85.3% acc., AUC 0.95	Metadata graph modeling	Complexity (3× latency vs. MLP), limited deployability

Cross-cutting analysis:

This synthesis highlights three systematic findings:

Finding 1 — Performance Progression: A clear progression from unimodal (72–73%) → simple multimodal (85%) → sophisticated multimodal (85–88%), validating the diagnostic value of clinical metadata.

Finding 2 — Performance–Complexity Trade-off: The most complex architectures (Multitask, GNN) are often the most challenging to deploy in real time on

lightweight infrastructure.

Finding 3 — Rarity of Operational Deployment: In the academic literature examined, virtually all studies conclude at offline evaluation without producing a complete, deployable software system (API + Application + CI/CD).

2.6.6 Formal Formulation of the Research Gap

Problem Statement

Academic studies in multimodal dermatological diagnosis (2018–2023) have convincingly demonstrated that integrating clinical metadata into CNN models significantly improves predictive performance (+3 to +7 accuracy points, +0.03 to 0.05 AUC). However, these systems generally remain confined to controlled research environments, creating an engineering gap between algorithmic research and assisted clinical practice.

Multidimensional Gap Analysis

Dimension 1: Architectural Gap

What is missing:

- Robust, documented REST API backend (OpenAPI/Swagger)
- Intuitive cross-platform mobile application
- Scalable cloud infrastructure (auto-scaling, load balancing)

Why it is critical: A PyTorch `.pth` model checkpoint does not constitute a usable system. Without an API and application, the model remains inaccessible to clinicians.

Dimension 2: Performance–Deployability Gap

What is missing:

- Explicit trade-off between complexity and latency
- Optimization for production constraints (CPU, RAM, response time)
- Realistic evaluation (network latency, user variability)

Why it is critical: Academic research frequently optimizes pure predictive performance without considering inference latency. A deployed system must achieve a balance: accuracy + speed + operational cost.

Dimension 3: Telemedicine Alignment Gap

What is missing:

- Telemedicine dataset (smartphone images)
- User interface tailored to patients (non-experts)

- Complete workflow: capture → questionnaire → result → action

Why it is critical: Most studies rely on professional dermoscopic images. However, real-world telemedicine relies on smartphone images of variable quality.

Scientific Formulation of the Gap

RESEARCH GAP:

There exists a systematic gap between multimodal architectures performing well in controlled academic environments and operational systems deployable under real-world telemedicine conditions, characterized by the absence of an integrated engineering approach combining:

1. An ML architecture optimized for production constraints
2. A complete software infrastructure (API + Mobile + MLOps)
3. Evaluation under realistic telemedicine conditions

This gap constitutes a major barrier to the clinical adoption of dermatological AI, despite the technical maturity of the underlying models.

2.6.7 Positioning of MedAssist AI

The present project explicitly aims to bridge this gap through a holistic approach structured around four pillars:

Pillar 1 — Pragmatic ML Architecture: MedAssist AI adopts a *Gated Cross-Attention Fusion* architecture (EfficientNet-B3 + MLP) rather than overly complex approaches such as GNNs [20], following an explicit trade-off where operational deployability is paired with diagnostic robustness. This architecture integrates a cross-attention mechanism (*Cross-Attention*: Q=metadata, K=V=image patches) with an adaptive gating mechanism ($g \cdot \text{attended} + (1 - g) \cdot \text{img_global}$) and positional embeddings for visual patches. This pragmatic design ensures inference latency compatible with cloud/mobile constraints while remaining substantially more expressive than naive concatenation.

Pillar 2 — Complete Software Infrastructure: The system relies on a complete architecture integrating backend, frontend, and MLOps. The backend utilizes FastAPI (Python) [24] for an asynchronous REST API with auto-generated OpenAPI documentation, and leverages Docker containerization [39] ensuring portability across development and production environments. The mobile frontend, developed in Flutter (Dart) [25], provides cross-platform deployment (iOS + Android) with a native UI/UX, implementing a guided five-step workflow (image capture → metadata questionnaire

→ inference → result → recommendation). The prototype's MLOps infrastructure relies on experiment tracking via MLflow [22] (metrics, hyperparameters, models) and a continuous integration pipeline via GitHub Actions (syntax checks, automated Docker builds). Large-scale data versioning via DVC [21] is planned as an industrialization roadmap item for full-scale deployment.

Pillar 3 — Realistic Telemedicine Evaluation: Evaluation relies on a multi-source unified dataset (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17]) combining dermoscopic images and smartphone photographs, with 7 standardized clinical metadata variables. Metrics follow a multidimensional approach combining ML performance (Accuracy, Macro and Weighted F1-Score, per-pathology Precision and Recall with class-specific adaptive thresholds θ_c), system performance (end-to-end nominal latency and throughput), and robustness under load constraints.

Pillar 4 — Documentation and Reproducibility: The project ensures reproducibility and clinical adoption through comprehensive documentation including architectural specifications, interactive OpenAPI/Swagger API documentation, a unified preprocessing pipeline, and reproducible environments (Docker, pinned requirements, fixed random seeds).

2.7 Chapter Conclusion

This chapter traced the evolution of AI-assisted dermatological diagnosis, revealing remarkable yet unfulfilled progress toward clinical adoption. The chronological analysis highlighted three phases: the technical foundations (2012–2016) with deep CNNs and Transfer Learning; the unimodal era (2017–2018) demonstrating that CNNs match dermatologists but revealing four limitations (metadata omission, vulnerability to distribution shifts, lack of explainability, ethnic bias) [7, 8]; and the multimodal transition (2018–2023) with enriched datasets [14–17] and sophisticated architectures [9, 10, 20] delivering predictive gains (+3–7% in accuracy).

The comparative analysis reveals a systematic paradox: as performance improves and complexity escalates, systems increasingly diverge from operational deployability. An examination of deployment attempts [11] identified a multi-dimensional structural gap: the absence of an integrated backend/mobile infrastructure, optimization conducted without regard for latency/cost, and a lack of alignment with real-world telemedicine conditions.

MedAssist AI aims to bridge this gap through a holistic approach: a *Gated Cross-Attention Fusion* architecture (EfficientNet-B3 + MLP), trained on a multi-source unified dataset (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17]) with pre-training on HAM10000 [14], a complete software infrastructure (FastAPI + Flutter), and fully reproducible documentation. The primary contribution establishes a reproducible

methodological template, paving the way for the clinical adoption of multimodal dermatological AI. The following chapter presents the detailed system design and methodology of MedAssist AI.

Chapter 3

MedAssist AI System Design

3.1 Introduction

This chapter presents the comprehensive design process of the MedAssist AI platform, directly addressing the architectural gap identified in Chapter 2: the absence of a solution capable of delivering the diagnostic performance of multimodal models through an accessible mobile interface.

The progression is **strictly top-down**: proceeding from the global system vision (actors, requirements, architecture) down to technical components, the AI model, the API, and the MLOps lifecycle. Each section prepares and motivates the subsequent one, forming a coherent and traceable decision chain.

3.2 System Requirements

Before defining the architecture, we specify the functional capabilities of the system and the non-functional quality constraints it must satisfy. These requirements form the baseline against which architectural and design decisions are evaluated.

3.2.1 Functional Requirements

Table 3.1 summarizes the primary functional requirements expected of the MedAssist AI system:

Table 3.1: MedAssist AI functional requirements

ID	Priority	Description
EF-01	Essential	The system must enable the capture or import of a skin lesion image from the mobile application.
EF-02	Essential	The system must collect up to 7 patient clinical metadata features (age, sex, anatomical site, growth, bleeding, diameter, personal history).
EF-03	Essential	The system must produce a prediction across 6 diagnostic classes with associated probabilities for each class.
EF-04	Essential	The system must categorize each result into four clinical risk levels: LOW, MODERATE, HIGH, and CRITICAL.
EF-05	Important	The system must provide the capability to run inference with 8× Test-Time Augmentation (`use_tta=true`) to maximize accuracy.
EF-06	Important	The API must expose a health check endpoint (`/health`) and a model information endpoint (`/model-info`).
EF-07	Important	The system must display a medico-legal disclaimer and generate a downloadable PDF report.

3.2.2 Non-Functional Requirements

The technical and operational constraints of the project are detailed in Table 3.2:

Table 3.2: MedAssist AI non-functional requirements

ID	Category	Description and Target
ENF-01	Performance	API latency strictly below 2 seconds at the 95th percentile ($p_{95} < 2\,\text{s}$).
ENF-02	Availability	Minimum throughput of 10 requests/second without performance degradation; resilience to 50 concurrent requests.
ENF-03	Portability	The mobile application must be compilable for Android and iOS from a single codebase (Flutter).
ENF-04	Security	All communications must be encrypted via HTTPS/TLS 1.3; no patient data must be persisted on the server.
ENF-05	GDPR Compliance	Data processing strictly in volatile memory (RAM); data minimization principle applied (Art. 5 GDPR [23]).
ENF-06	Maintainability	Every experiment must be trackable via MLflow, and code integration must be ensured through a CI/CD pipeline (GitHub Actions).
ENF-07	Deployability	The server must be containerized (Docker) to guarantee parity between development and production environments.
ENF-08	Footprint	The Flutter APK size (release build) must not exceed 30 MB.

Based on these requirements, we next define system actors and their operational workflows through use case modeling.

3.3 Architectural Design

Architectural design translates requirements into a structural organization of the system. It addresses three fundamental questions: *who* interacts with the system? *in what sequence*? and *under what layered organization*? These three questions correspond respectively to the three UML diagrams presented below: use case, sequence, and class diagrams.

3.3.1 Use Case Diagram

Figure 3.1 identifies the system actors and their interactions with the exposed functionalities. Two actors are defined: the **Practitioner** (physician using the mobile application) and the **Administrator** (responsible for the model lifecycle).

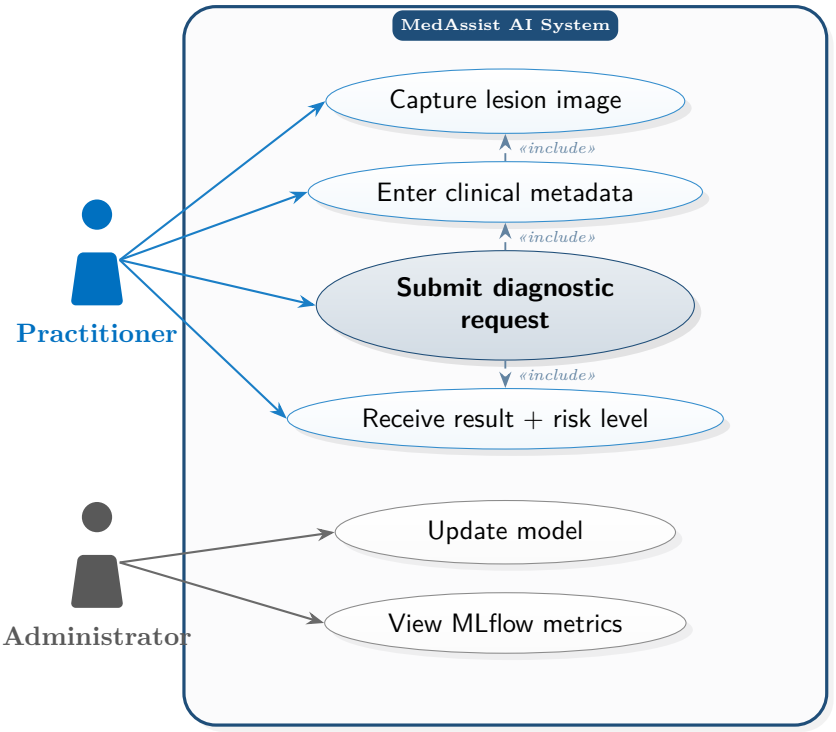


Figure 3.1: MedAssist AI use case diagram

The use cases reveal a natural separation of responsibilities: collection (mobile), processing (server), and persistence (data/model). This observation directly motivates the choice of a three-tier architecture.

3.3.2 Three-Tier Architecture: Component View

The three-tier architecture [40] shown in Figure 3.2 organizes the system into three distinct layers:

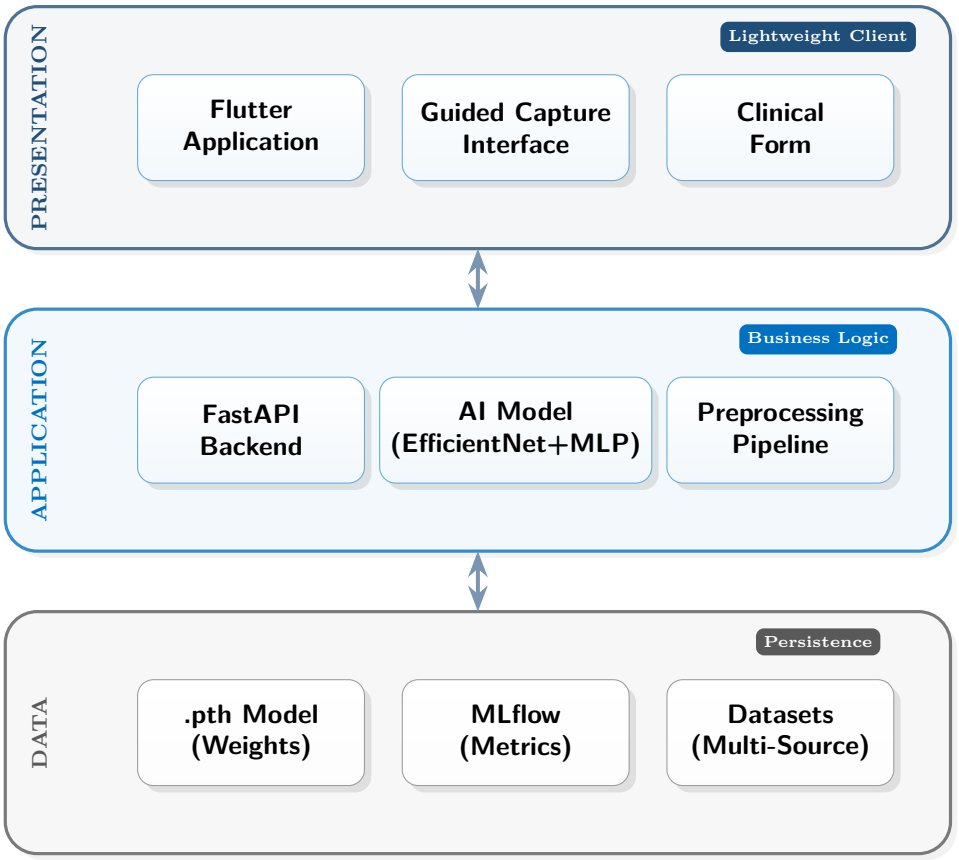


Figure 3.2: Three-tier software architecture

The software architecture is structured across three distinct layers:

- **Tier 1 Presentation:** A lightweight client dedicated to the user interface, without any embedded inference logic.
- **Tier 2 Application:** The central layer orchestrating data validation, preprocessing, AI model inference, and response formatting.
- **Tier 3 Data:** Responsible for storing model weights, managing unified datasets, and maintaining metric traceability via MLflow.

Having defined the system tiers, we examine the dynamic communication flow during a diagnostic query.

3.3.3 Sequence Diagram: Request Flow

Figure 3.3 details the execution sequence of a complete diagnostic request, tracing the interaction from image capture to the delivery of results.

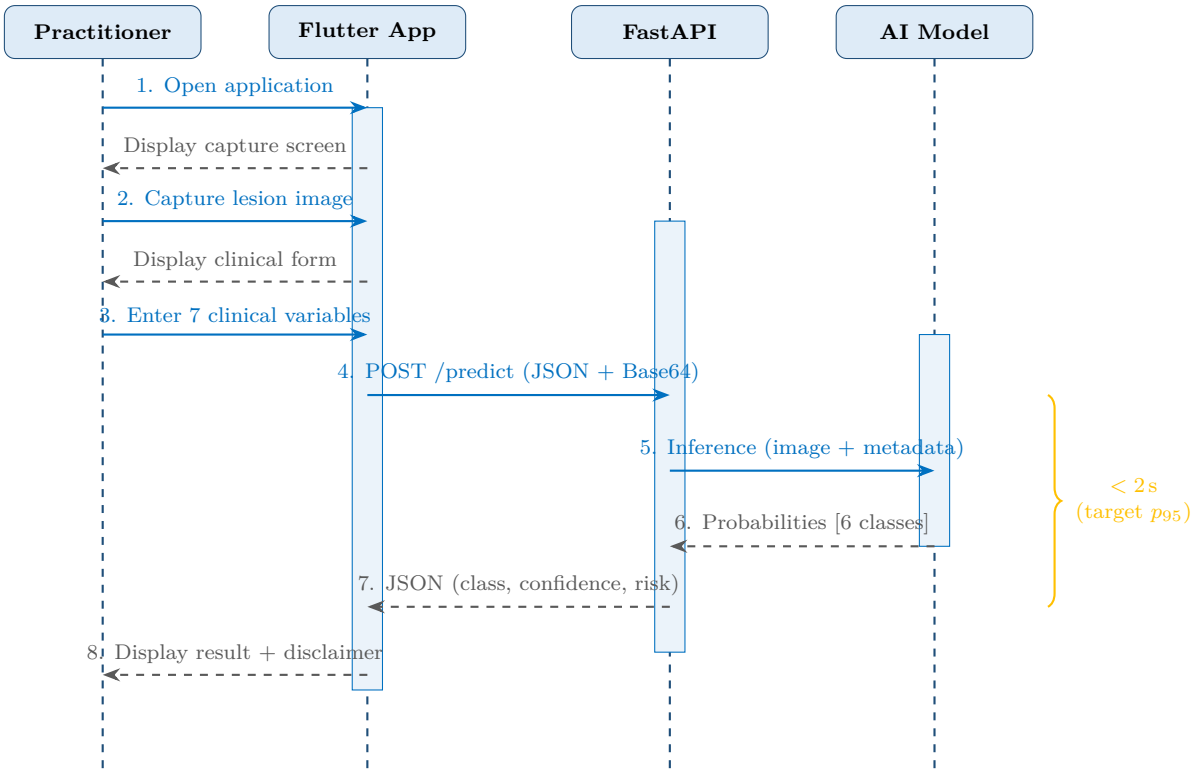


Figure 3.3: Sequence diagram: Diagnostic request flow

This flow confirms that all intelligent processing (steps 5–6) is confined to the server, ensuring that the mobile application remains a lightweight client. The constraint $p_{95} < 2\,\text{s}$ (ENF-01) bounds the technical choices of both the model and the API.

With the sequence clarified, the next step is to formalize the domain entities and relationships manipulated by the API—the role of the class diagram.

3.3.4 Class Diagram: Domain Model

The diagram in Figure 3.4 formalizes the entities exchanged between the mobile application and the API, as well as their relationships.

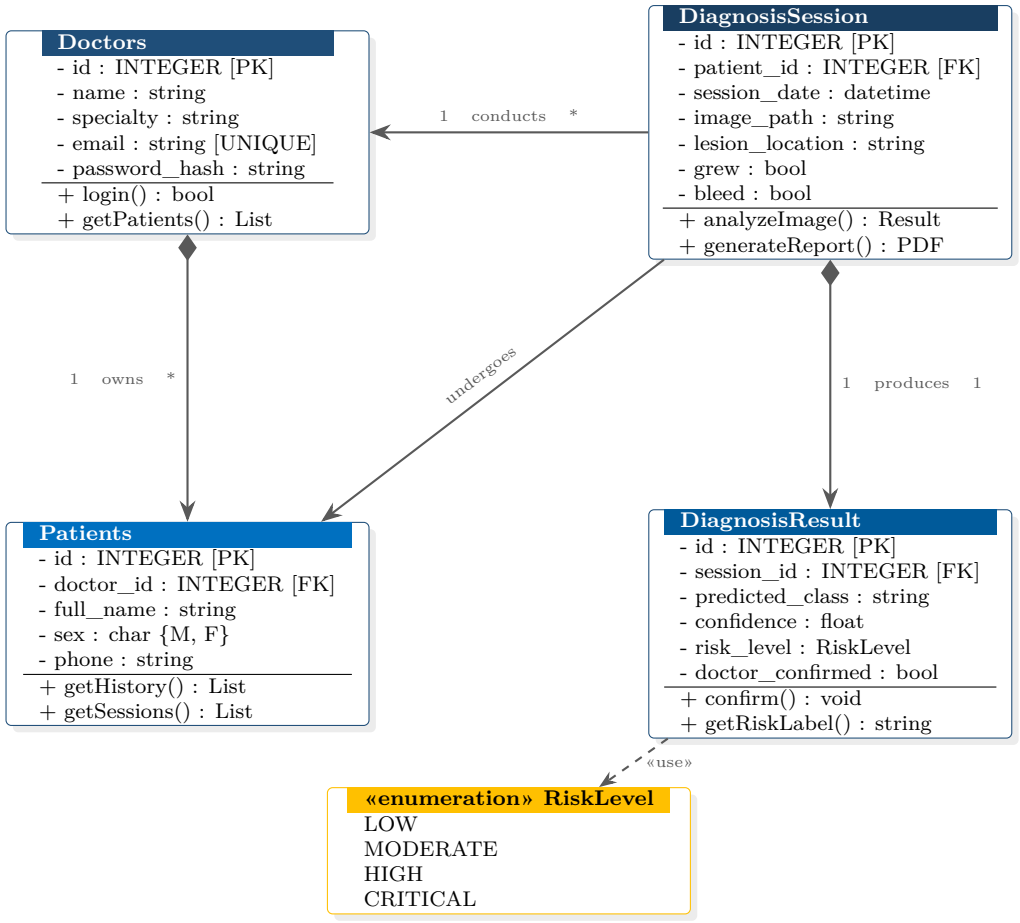


Figure 3.4: Class diagram: MedAssist AI system

Following the architectural specification, the next section describes the preparation and curation of datasets used to train the multimodal network.

3.4 Data Preparation

The MedAssist AI (V6.0) pipeline relies on a **multi-source unified dataset** of 6,581 standardized clinical images aggregated from three databases: PAD-UFES-20, MCR-SL, and a selective subset of ISIC 2019, while HAM10000 serves for initial backbone pre-training. Combining these sources counteracts single-dataset class imbalance, broadens representation across Fitzpatrick phototypes (IV–VI), and improves cross-domain generalization under realistic telemedicine conditions.

3.4.1 Class Distribution and Patient-Level Split

To prevent data leakage, dataset partitioning into training, validation, and test splits is executed strictly at the **patient level** (*Patient-Level Stratified Split*) based on `patient_id`. We performed an exhaustive search across **10 stratified folds** to identify the split that minimizes class disparity in the independent test set (measured by the

ratio $\text{classes}_{\max}/\text{classes}_{\min}$). **Fold 02** yielded the best balance (ratio = 1.66) among the 10 evaluated partitions. This partition allocates 75.9% of images to training (4,993 images prior to resampling), 14.2% to validation (934 images), and 9.9% to the independent test set (654 images), with zero patient overlap across splits.

Table 3.3 summarizes the distribution of the 6 diagnostic classes in the unified dataset.

Table 3.3: Diagnostic class distribution in the unified MedAssist AI dataset (V6.0)

Code	Pathology	Nature	Images (Total)	Support (Test)
ACK	Actinic keratosis	Precancerous	1,325	130
BCC	Basal cell carcinoma	Malignant	845	92
MEL	Malignant melanoma	Malignant	620	65
NEV	Melanocytic nevus	Benign	1,522	149
SCC	Squamous cell carcinoma	Malignant	863	90
SEK	Seborrheic keratosis	Benign	1,406	128
	Overall Total		**6,581**	**654**

3.4.2 Preprocessing Pipeline and Metadata Masking

To mitigate acquisition artifacts and standardize inputs, images undergo a five-stage preprocessing pipeline:

1. **Lanczos Interpolation (256×256 px):** Resamples the image while preserving edge sharpness and lesion boundary details.
2. **Shades-of-Gray Color Constancy ($p = 6$):** Corrects chromatic shifts induced by diverse lighting and dermatoscope sources [33].
3. **DullRazor Hair Removal:** Applies a morphological Black-Hat filter (17×17) followed by median inpainting (15×15) to remove occluding hair artifacts [41].
4. **Contrast Limited Adaptive Histogram Equalization (CLAHE):** Enhances local contrast on the luminance channel L^* of the CIE Lab color space (`clipLimit=2.0`, 8×8 grid).
5. **ImageNet Normalization:** Scales input channels using standard pre-training parameters ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).

For tabular clinical features, we select 7 variables aligned with the ABCDE diagnostic criteria. When specific patient fields are unavailable, missing values are imputed (median for continuous variables, mode for categorical variables) and paired with an

explicit **binary presence mask (`meta_mask`)** $\mathbf{M} \in \{0,1\}^7$ provided directly to the model.

Table 3.4: Clinical variables selected for the model and presence mask (`meta_mask`)

#	Variable	Type	Description / Clinical Justification
1	`age`	Numerical	Patient age in years (normalized with StandardScaler) [2]
2	`gender`	Binary	Biological sex converted to binary (1: Male, 0: Female / Unknown) [42]
3	`grew`	Binary	Recent evolution or growth (ABCDE criterion **E**) [35]
4	`bleed`	Binary	Spontaneous lesion bleeding (sign of ulceration) [3]
5	`diameter_1`	Numerical	Maximum lesion diameter in mm (ABCDE criterion **D**) [35]
6	`skin_cancer_history`	Binary	Personal history of skin cancer [2]
7	`elevation`	Binary	Palpable elevation or contour of the lesion (ABCDE criterion **E**) [35]

3.5 Multimodal Model Architecture (V6.0)

The **MedAssist AI V6.0** network contains **12,616,117 parameters** and employs a clinical-guided cross-attention architecture (*Gated Cross-Attention Fusion*).

3.5.1 Visual Branch: EfficientNet-B3, Projection, and Auxiliary Head

The visual feature extractor uses a pre-trained **EfficientNet-B3** backbone [12]:

- **Auxiliary Classification Head:** A parallel branch pools feature maps using **Generalized Mean (GeM) Pooling** ($p = 3.0$) [43]:

$$f_{\text{GeM}} = \left(\frac{1}{|X|} \sum_{x \in X} x^p \right)^{1/p}$$

  followed by `Linear(1536→256)` → `GELU` → `Dropout(0.3)` → `Linear(256→6)`. This auxiliary head is assigned a 30% weight in the total training loss ($0.7 \cdot L_{\text{main}} + 0.3 \cdot L_{\text{aux}}$), serving as a deep supervision regularizer.

- **Visual Patch Projection:** Deep convolutional features are projected through `Conv2d(1536→512)` → `BatchNorm` → `GELU` → `Conv2d(512→256)` → `BatchNorm`

and downsampled with `AdaptiveAvgPool2d(7)`, producing a spatial grid of $7 \times 7 = \mathbf{49}$ visual patches of dimension 256.

- **Positional Embeddings:** A learnable positional encoding tensor $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{1\times 49\times 256}$ is added to the patch embeddings to preserve spatial lesion topology.

3.5.2 Clinical Branch: MLP with Masking and LayerNorm

The clinical branch processes the 7 patient variables masked by the presence vector $\mathbf{x}_{\text{masked}} = \mathbf{x} \odot \mathbf{M}$:

$$\text{Linear}(7 \to 128) \to \text{LayerNorm}(128) \to \text{GELU} \to \text{Dropout}(0.30)$$

$$\to \text{Linear}(128 \to 64) \to \text{LayerNorm}(64) \to \text{GELU} \to \text{Dropout}(0.35) \to \text{Linear}(64 \to 64)$$

This produces a 64-dimensional metadata embedding, which is then projected to 256 dimensions ($\mathbf{Q}_{\text{meta}}$) to query the visual patches during cross-attention.

3.5.3 Gated Cross-Attention Fusion, Classifier, and Training Protocol

Multimodal fusion is carried out through Multi-Head Cross-Attention (4 heads, $d_k = 256$) [13], where the clinical metadata embedding functions as the Query ($\mathbf{Q}$) and the 49 visual patches serve as Keys and Values ($\mathbf{K}, \mathbf{V}$):

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

A learnable scalar gate $g \in [0, 1]$ balances the attended representation against the global visual embedding:

$$\mathbf{z}_{\text{fused}} = g \cdot \text{attended} + (1 - g) \cdot \text{img_global}$$

The final classification head consists of:

$$\text{Linear}(256 \to 256) \to \text{LayerNorm}(256) \to \text{GELU} \to \text{Dropout}(0.55)$$

$$\to \text{Linear}(256 \to 128) \to \text{LayerNorm}(128) \to \text{GELU} \to \text{Dropout}(0.45) \to \text{Linear}(128 \to 6)$$

Loss formulation. To address severe class imbalance, the training objective combines **Weighted Focal Loss** ($\gamma = 2.0$) [44] with **Label Smoothing** ($\epsilon = 0.05$) [45].

Two-phase training schedule. The optimization workflow follows the two-stage protocol outlined in Table 3.5, employing AdamW optimization [46], CosineAnnealingWarmRestarts scheduling [47], and Stochastic Weight Averaging (SWA) [48].

Table 3.5: Two-phase progressive training protocol (V6.0)

Parameter	**Phase 1 (Pre-training)**	**Phase 2 (Unified Fine-Tuning)**
Image resolution	128×128 px	256×256 px
Epochs	55 epochs (5 frozen + 50 unfrozen)	45 epochs (overall total 36–80)
Backbone unfreezing	Starting from epoch 6	Unfrozen from the start
Learning rate	Backbone: 10^{-5} / Head: 10^{-4}	Backbone: 5×10^{-6} / Head: 5×10^{-5}
Scheduler	CosineAnnealingWarmRestarts ($T_0 = 10$) [47]	CosineAnnealingWarmRestarts ($T_0 = 10$) [47]
SWA (Stochastic Weight Averaging)	No	Yes (starting from epoch 15 of Phase 2) [48]
Validation TTA	TTA×4 (from epoch 5)	TTA×4 (from epoch 5)
Final test evaluation	—	**TTA×8 (8 geometric augmentations)**

3.6 REST API Specification

The REST API constitutes the single entry point of the system. It receives data from the mobile application, coordinates inference, and returns a structured result.

Endpoints. The API exposes three primary endpoints (Table 3.6). The diagnostic endpoint (`POST /api/v1/predict`) accepts a `multipart/form-data` payload containing the image file, the 7 clinical form fields, and an optional `use_tta` flag.

Table 3.6: REST API endpoint specifications

Method	**Endpoint**	**Auth.**	**Description**
POST	`/api/v1/predict`	None (MVP)	Multimodal inference (Image + Form-Data + `use_tta`)
GET	`/api/v1/health`	None	Health check for server status and PyTorch model
GET	`/api/v1/model-info`	None	Architecture details and diagnostic classes

Clinical risk mapping. Following multimodal inference, the system derives a clinical risk level (*Clinical Risk Level*) combining the predicted pathology and the confidence score. This categorization aims to streamline medical triage in telemedicine by immediately directing the practitioner toward appropriate management (from simple periodic self-monitoring for benign lesions to urgent referral with biopsy for melanomas).

Table 3.7 specifies the decision rules and recommendations for each risk category.

Table 3.7: Decision matrix and clinical risk level mapping

Risk Level	Color Code	Associated Classes	Rationale and Clinical Recommendation
LOW	**#22c55e**	NEV, SEK	Benign lesions (*benign*); periodic self-monitoring.
MODERATE	**#f59e0b**	ACK	Precancerous lesion (*pre-malignant*); preventive follow-up (*watch & wait*).
HIGH	**#ef4444**	BCC, SCC	Malignant pathologies (*malignant*); treatment required, consultation within 2 weeks.
CRITICAL	**#7c3aed**	MEL	Aggressive cancer (*aggressive*); urgent referral (*urgent referral*), immediate biopsy.

Mobile client workflow. The Flutter application guides the user through a five-step screen sequence designed for rapid clinical use (< 2 minutes of data entry): **Home** (consent) $\rightarrow$ **Guided Capture** (overlay + quality validation) $\rightarrow$ **Clinical Form** (7 variables, < 30 s) $\rightarrow$ **Loading** (POST /predict) $\rightarrow$ **Results** (color code + medico-legal disclaimer).

We next detail the operational MLOps lifecycle supporting continuous integration and model tracking.

3.7 MLOps Pipeline, CI/CD, and Traceability

The MLOps pipeline guarantees reproducibility, traceability, and scalability across development and production environments [49]. Within the MedAssist AI MVP prototype, the infrastructure is built upon three complementary components:

Experiment Tracking with MLflow [22]: Integrated into both training stages to record hyperparameters (learning rate schedules, T_0, weight decay values, focal loss weights) and epoch metrics (loss, macro F1, per-class F1), providing full iteration history.

Docker Containerization [39]: The FastAPI backend and PyTorch runtime are containerized in a Linux image configured for CPU execution, preventing environmental drift between local machines and target servers.

Automated CI via GitHub Actions: Configured through `.github/workflows/ci.yml` to execute static linting (`flake8`) and verify Docker image compilation on every code push or pull request.

Startup industrialization perspectives. Large-scale data artifact versioning via DVC [21] and automated retraining upon data drift represent planned extensions for the market launch phase.

3.8 Evaluation Protocol

System validation evaluates both machine learning performance (Table 3.8) and software engineering metrics (Table 3.9), verifying compliance with requirements ENF-01 through ENF-04.

Table 3.8: Target metrics and validated results on the unified benchmark (V6.0)

AI Metric	**Design Target**	**Unified Test Result (654 samples)**
Accuracy	$> 70.0\%$	**74.31%**
Macro F1-Score	> 0.7000	**0.7315**
Weighted F1-Score	> 0.7000	**0.7373**
F1-Score BCC (Basal cell carcinoma)	> 0.8000	**0.8438**
F1-Score NEV (Melanocytic nevus)	> 0.8000	**0.8256**
F1-Score MEL (Malignant melanoma)	> 0.6500	**0.6780**
F1-Score SCC (Squamous cell carcinoma)	> 0.6500	**0.6629**
F1-Score SEK (Seborrheic keratosis)	> 0.6500	**0.6960**
F1-Score ACK (Actinic keratosis)	> 0.6500	**0.6825**

Table 3.9: Software system evaluation metrics

System Metric	**Method**	**Target / Result**
API latency (p_{95})	CPU inference with 8× TTA	$< 2\,\text{s}$ (measured: $1.32\,\text{s}$)
Throughput	Docker container 3.5GB RAM	$> 10\,\text{req/s}$
Stress test	50 concurrent requests	Zero failures
Flutter APK size	Android release build	$< 30\,\text{MB}$

3.9 Conclusion

This chapter conducted a rigorous top-down design: from functional and non-functional requirements to the evaluation protocol, encompassing the three UML diagrams, the three-tier architecture, data preparation, the multimodal model, the REST API, and the MLOps pipeline. Each decision prepares and motivates the subsequent one.

The strict separation between a lightweight client (Flutter) and an intelligent server (FastAPI + *Gated Cross-Attention Fusion* model) directly resolves the paradox identi-

fied in Chapter 2: deploying an effective multimodal model without sacrificing mobile accessibility. The following chapter translates this design into a deployable technical implementation.

Chapter 4

Technical Implementation

4.1 Introduction

Following the definition of the system architecture and design in Chapter 3, this chapter presents the concrete implementation of the MedAssist AI platform. We successively describe the development environment, the project organization and modular directory structure, and the implemented components: the artificial intelligence model, the REST API, and the Flutter mobile application. The chapter concludes with functional testing and theoretical capacity validation.

4.2 Development Environment

4.2.1 Hardware Environment

Code development, the FastAPI backend, and the mobile application were executed on a local workstation, whereas intensive multimodal deep learning training and fine-tuning were performed on Cloud infrastructure (Google Colaboratory with GPU acceleration). Mobile validation was conducted on a physical test smartphone (Table 4.1).

Table 4.1: Hardware specifications across local, cloud, and mobile environments

Component	**Local Development (PC)**	**Cloud Training (Colab)**	**Mobile Testing**
Model / Platform	Laptop PC (Intel x64)	Google Colaboratory	Xiaomi Redmi / Poco
Processor	Intel Core i5-8250U @ 1.60GHz	Intel Xeon (2 vCPUs)	Octa-core ARM
RAM	8.00 GB DDR4	12.7 GB RAM	8.00 GB
Storage	256 GB SSD	100 GB Cloud Disk	128 GB
GPU Accelerator	Intel UHD Graphics 620	NVIDIA Tesla T4 (16 GB VRAM)	Adreno / Mali
Operating System	Microsoft Windows 11 / 10	Ubuntu 22.04 LTS (Linux)	Android 13 / 14

4.2.2 Software Environment

The software environment relies on a complementary set of engineering tools covering AI development, FastAPI backend service, mobile engineering, and MLOps lifecycle management.

Python 3.11 The primary programming language of the project, employed across the data engineering pipeline, neural network training, and the REST API. Its extensive ecosystem of scientific computing libraries makes it the de facto standard for artificial intelligence engineering.

PyTorch 2.1 The core deep learning framework selected for its dynamic computational graph (*eager execution*), architectural flexibility, and mature production deployment ecosystem via `torch.save` and `torchvision`.

FastAPI 0.104 A modern, asynchronous Python web framework used to construct the REST API. Its native OpenAPI/Swagger schema generation and strict data validation via Pydantic make it optimal for medical APIs that demand rigorous schema validation of clinical inputs.

Flutter 3.16 / Dart A cross-platform software development kit utilized for the client mobile application. It enables Android and iOS applications to be built from a single codebase, with near-native performance through the Skia-based rendering engine used by this Flutter version.

Docker 24.0 Containerization platform ensuring runtime reproducibility and portability of the FastAPI backend between local development and production deployment environments.

MLflow 2.8 An open-source MLOps platform employed for experiment tracking, systematically logging hyperparameters, evaluation metrics, and trained model artifacts across training runs.

GitHub Actions CI/CD An automated continuous integration service executing automated validation test suites and building the Docker container image upon every code commit.

Visual Studio Code The integrated development environment (IDE) utilized for Python and Dart development, supplemented by the Pylance, Flutter, and Docker extensions.

4.3 Modular Project Organization and Environments

The implementation of the MedAssist AI platform is structured around a distributed modular architecture designed to decouple massive data engineering, deep model training, and lightweight telemedicine deployment. The project is organized into three interconnected environments:

4.3.1 AI Pipeline Organization and Cloud Persistence (Google Drive / Colab)

The AI engineering and experimental training infrastructure is hosted on Google Colaboratory and synchronized with persistent Google Drive storage (`MyDrive/MedAssist_AI/`):

- **Persistent raw datasets storage (`dataset/`, `dataset_isic2019/`, `dataset_mcr_sl/`)**: Stores heterogeneous data sources (PAD-UFES-20, ISIC 2019, and MCR-SL) alongside compressed backup archives ensuring data integrity.
- **Processed data and unified metadata (`data/processed/`)**: Contains patient-level stratified dataset partitions (`train_metadata.csv`, `val_metadata.csv`, `test_metadata.c` feature encoding configurations, and serialized scalers (`scaler.pkl`).
- **Checkpoint management (`models/`)**: Archives model weights across training stages (`ham10000_backbone_b3.pth` for transfer pre-training, `phase1_best.pth` for classifier warmup, and `best_swa.pth` for final SWA inference).
- **Experiment traceability (`results/mlruns/`)**: Local MLflow tracking directory recording hyperparameters, training trajectories, and evaluation metrics for Phase 1 and Phase 2.
- **Fast SSD caching mechanism on Colab (`/content/fast_images/`)**: During training, preprocessed images (Shades-of-Gray, DullRazor, CLAHE) are saved as preprocessed PNG images directly on the local SSD of the Colab instance, eliminating I/O bottlenecks and maximizing GPU throughput.

4.3.2 Backend Inference Server Architecture (FastAPI)

The inference server (`backend_App_MedAssist_AI/`) is strictly decoupled from the training environment and dedicated to real-time serving:

- **REST Routing (`app/main.py`, `app/api/`)**: Exposes OpenAPI/Swagger compliant endpoints (`POST /api/v1/predict`, `GET /health`, `GET /model-info`).

- **DTO Data Contracts (`app/models/schemas.py`)**: Enforces strict schema validation of incoming payloads (7 clinical variables, multipart form types) and structures output responses with clinical risk levels via Pydantic v2.
- **Containerization and CI/CD (`Dockerfile`, `.github/workflows/ci.yml`)**: Encapsulated container deployment ensuring dedicated CPU execution with 3.5 GB of RAM and automated testing on every code integration.

4.3.3 Mobile Application Architecture (Flutter)

The client frontend (`app_PFE_frontend/`) applies the modular *Feature-First* architectural pattern, guaranteeing structural independence and testability for every feature:

- **Functional Modules (`lib/modules/`)**: Encapsulates the 10 independent domain features:
  - `splash/` and `onboarding/`: Welcome view, patient informed consent, and medico-legal disclaimer.
  - `home/`: Main clinical dashboard to initiate a new diagnostic consultation.
  - `camera/`: Guided photographic acquisition with a circular centering overlay and real-time image quality assessment (sharpness, illumination).
  - `clinical_form/`: Ergonomic data entry for the 7 clinical variables (age slider, binary/categorical selectors).
  - `analysis/`: Animated loading view during asynchronous inference execution.
  - `result/`: Hierarchical diagnostic presentation, color-coded risk indicator, detailed class probabilities, and clinical PDF report generation.
  - `history/`: The `history/` module ensures local persistence of past consultations via `GetStorage`.
  - `education/` and `settings/`: Dermatological prevention educational sheets and network configuration parameters.
- **Data and Network Layer (`lib/data/`)**: Data model classes (`models/`), Dio-based HTTP/REST network provider (`providers/`), and clinical PDF report repository (`repositories/`).
- **Cross-Cutting Core (`lib/core/`, `lib/global_widgets/`)**: System constants, medical design system theme, and reusable UI components (*RiskBadge*, action buttons).

Table 4.2 summarizes the modular decomposition, detailing architectural responsibilities, mobilized software technologies, and primary roles across all packages.

Table 4.2: Modular decomposition matrix and software responsibilities

Subsystem	Package	Key Files	Technologies	Primary Role
Frontend	`lib/modules/*`	`camera/, form/, result/`	Flutter / Dart	Reactive UI, guided acquisition, 7-variable clinical form, triage, and PDF export.
	`lib/data/*`	`providers/, repositories/`	Dio / Pdf	Multipart REST requests, cache management, and clinical report generation.
	`lib/core/*`	`theme/, constants/`	Material 3	Medical theme design system, risk badges, and application routing.
Backend	`app/api`	`endpoints/ predict.py`	FastAPI [24]	RESTful routes (/predict, /health), HTTP status code management, and OpenAPI.
	`app/core`	`config.py, security.py`	Pydantic	Configuration settings (.env), structured logging, and CORS handling.
	`app/models`	`schemas.py`	Pydantic v2	DTO contracts and strict validation of the 7 clinical variables.
	`app/services`	`model_service.py`	PyTorch [6]	CPU inference pipeline, TTA×8, and clinical risk escalation engine.
AI Pipeline	`data_pipeline`	`02_preprocessing .ipynb`	OpenCV / NumPy [33, 41]	Shades-of-Gray normalization, DullRazor, and SSD caching (PNG).
	`model_arch`	`architecture.py`	Timm [30] / PyTorch	EfficientNet-B3 [12], GeM [43], and Gated Cross-Attention [13].
	`training`	`04a_b_train .ipynb`	MLflow [22] / SWA [48]	Two-phase training, Focal Loss [44], and CutMix [50].
DevOps	`devops`	`.github/ci.yml, Dockerfile`	Actions / Docker [39]	Automated CI, flake8 linting, and server containerization.

4.4 AI Model Presentation and Mathematical Formalization

4.4.1 Preprocessing Pipeline and Digital Normalization

To mitigate heterogeneous acquisition artifacts across dermoscopic and clinical photographs, the implementation in `02_preprocessing.ipynb` and the `preprocessing_service.py` service employs a deterministic visual engineering pipeline:

1. Minkowski Color Constancy (Shades-of-Gray, $p = 6$) [33]: To neutralize chromatic shifts introduced by uncalibrated smartphone camera sensors, global scene illumination across each color channel $c \in \{R, G, B\}$ is estimated using the L_p norm with $p = 6$:

$$\mu_{c,p} = \left(\frac{1}{|\Omega|} \sum_{\mathbf{x} \in \Omega} I_c(\mathbf{x})^p \right)^{\frac{1}{p}} \tag{4.1}$$

where Ω denotes the set of pixels in the 256×256 input image. The normalized image $\hat{I}_c(\mathbf{x})$ is subsequently obtained via chromatic realignment:

$$\hat{I}_c(\mathbf{x}) = \frac{I_c(\mathbf{x})}{\mu_{c,p} \sqrt{3} \cdot \left(\sum_{k \in \{R,G,B\}} \mu_{k,p}^2 \right)^{-1/2}} \tag{4.2}$$

2. Morphological Hair Removal (DullRazor) [41]: Occlusive hair structures are isolated using a morphological *Black-Hat* transformation with a rectangular structuring element $\mathbf{K} \in \{0, 1\}^{17 \times 17}$:

$$\mathbf{B}(\mathbf{x}) = (\mathbf{I} \bullet \mathbf{K})(\mathbf{x}) - \mathbf{I}(\mathbf{x}) \tag{4.3}$$

where $\bullet$ represents the morphological closing operator. An intensity threshold ($\tau = 10$) generates the binary hair mask $\mathbf{M}_{hair}$, followed by inpainting via an adaptive 15×15 median filter.

3. Contrast-Limited Adaptive Histogram Equalization (CLAHE): The image is projected into the CIE $L^*a^*b^*$ perceptual color space. Contrast-limited adaptive histogram equalization is applied exclusively to the luminance channel L^* with a clip limit of 2.0 and an 8×8 tile grid, prior to reconversion into RGB space.

4.4.2 Clinical Metadata Branch and Availability Masking (`meta_mask`)

To manage the intrinsic heterogeneity of the 7 standardized clinical metadata variables (`age`, `gender`, `grew`, `bleed`, `diameter_1`, `skin_cancer_history`, `elevation`)

across the three multi-source datasets (PAD-UFES-20, ISIC 2019, MCR-SL), a **binary availability mask** $\mathbf{M} \in \{0,1\}^7$ is fed directly into the input of the MLP branch (`MLPBranch`).

Each component $M_j = 1$ indicates that clinical variable j was explicitly documented by the practitioner, whereas $M_j = 0$ indicates that the variable was missing and was statistically imputed (using the median for continuous variables and the mode for binary variables computed across the training partition). The direct masking operation is defined as:

$$\tilde{\mathbf{x}}_{meta} = \mathbf{x}_{meta} \odot \mathbf{M} \tag{4.4}$$

where $\odot$ denotes the Hadamard product (element-wise multiplication). Imputed values are thereby zeroed out and neutralized *before* entering the initial linear layer of the MLP network ($\mathbb{R}^7 \to \mathbb{R}^{128} \to \mathbb{R}^{64}$).

Unlike computationally heavy or iterative imputation techniques (such as k-NN [51], MICE, or generative variational autoencoders), this formulation is strictly deterministic, requires zero computational overhead, and preserves the fidelity of the clinical signal by preventing the model from learning spurious correlations over synthetic values.

Furthermore, this masking mechanism interacts synergistically with the gating fusion mechanism (Section 4.4.4): when all clinical variables are absent ($\mathbf{M} = \mathbf{0}$), $\tilde{\mathbf{x}}_{meta} = \mathbf{0}$, the resulting attention query is essentially neutral, and the adaptive gate g empirically attenuates toward zero ($g \to 0$). This achieves *graceful degradation* toward an essentially unimodal visual classification, with no inference failure and without requiring artificial conditional branching (`if/else`).

4.4.3 Visual Branch and Generalized Mean Pooling (GeM)

The visual branch extracts spatial representations using the deep convolutional network **EfficientNet-B3** [12]. The weights of the EfficientNet-B3 backbone are initially pretrained on HAM10000 and then transferred into the multimodal model prior to fine-tuning.

The terminal feature map $\mathbf{X} \in \mathbb{R}^{1536\times8\times8}$ is resampled to a spatial resolution of 7×7 via Adaptive Average Pooling, and subsequently projected into 49 visual patch embeddings $\mathbf{Z}_{img} \in \mathbb{R}^{49\times256}$ through two successive 1×1 convolutions with batch normalization (*BatchNorm*) and GELU activations.

In parallel, the auxiliary regularization head incorporates a **Generalized Mean (GeM) Pooling** layer [43] with a learnable exponent initialized at $p = 3.0$ and jointly optimized with the other network parameters:

$$\mathbf{f}_{GeM} = \left(\frac{1}{|\mathcal{X}|} \sum_{\mathbf{x}\in\mathcal{X}} \mathbf{x}^p \right)^{\frac{1}{p}} \tag{4.5}$$

For $p > 1$, the GeM operator differentiably sharpens salient activations associated with focal dermatological lesions relative to healthy surrounding skin, outperforming the oversmoothing tendency of standard Global Average Pooling ($p = 1$).

4.4.4 Gated Cross-Attention Fusion Architecture

The neural architecture implemented in the `MedAssistModel` (comprising 12,616,117 parameters) integrates the deep visual branch and the guided clinical branch via an adaptive cross-attention mechanism (Figure 4.1).

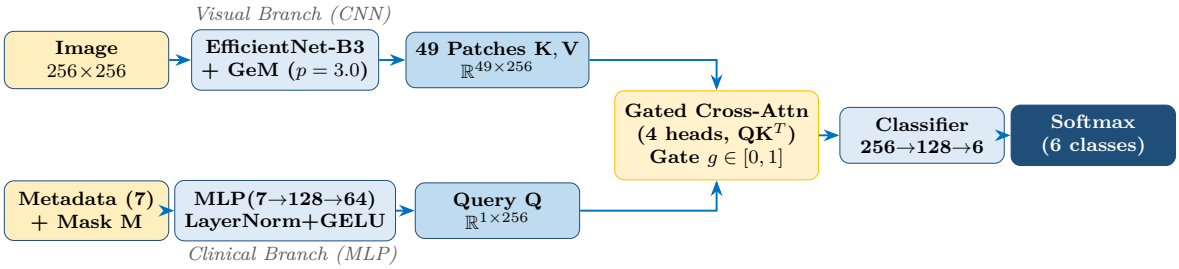


Figure 4.1: Implemented Gated Cross-Attention neural architecture (V6.0)

Mathematical Formulation of Cross-Attention Fusion: Let $\mathbf{z}_{meta} \in \mathbb{R}^{1\times256}$ represent the clinical embedding produced by the MLP branch, and let $\mathbf{Z}_{img} \in \mathbb{R}^{49\times256}$ represent the 49 visual patch vectors. Multi-head linear attention projections ($h = 4$ heads, key dimension $d_k = 64$) are defined by:

$$\mathbf{Q} = \mathbf{z}_{meta}\mathbf{W}_q \in \mathbb{R}^{1\times256}, \quad \mathbf{K} = \mathbf{Z}_{img}\mathbf{W}_k \in \mathbb{R}^{49\times256}, \quad \mathbf{V} = \mathbf{Z}_{img}\mathbf{W}_v \in \mathbb{R}^{49\times256} \tag{4.6}$$

For each head $i \in \{1, \ldots, 4\}$, the cross-attention matrix spatially weights the visual features conditioned on the patient's clinical profile:

$$\text{head}_i = \text{Softmax}\left(\frac{\mathbf{Q}_i\mathbf{K}_i^T}{\sqrt{d_k}}\right)\mathbf{V}_i \in \mathbb{R}^{1\times64} \tag{4.7}$$

The aggregated vector $\mathbf{f}_{attn} = [\text{head}_1 \,\|\, \ldots \,\|\, \text{head}_4]\mathbf{W}_o \in \mathbb{R}^{1\times256}$ is then fused with the global visual representation $\mathbf{f}_{img}^{avg} \in \mathbb{R}^{1\times256}$ through an adaptive residual gate $g \in [0, 1]$:

$$g = \sigma\left(\mathbf{W}_g[\mathbf{f}_{img}^{avg} \,\|\, \mathbf{f}_{attn}] + b_g\right) \tag{4.8}$$

$$\mathbf{f}_{fused} = g \cdot \mathbf{f}_{attn} + (1 - g) \cdot \mathbf{f}_{img}^{avg} \tag{4.9}$$

where σ denotes the sigmoid activation function, $[\cdot \,\|\, \cdot]$ vector concatenation, and $\mathbf{W}_g \in \mathbb{R}^{512\times1}$ a learnable projection vector. The resulting multimodal feature vector $\mathbf{f}_{fused}$ feeds into the terminal classification head (Linear($256 \to 128$) $\to$ GELU $\to$ Dropout(0.3) $\to$ Linear($128 \to 6$)) to yield the diagnostic probability distribution $\hat{\mathbf{y}} = \text{Softmax}(\mathbf{z}_{logits})$.

4.4.5 Regularization Functions and Test-Time Augmentation Inference

The modular implementation within the experimental notebooks and inference service leverages optimized training and evaluation routines:

- `apply_tta(images)`: Synthesizes 8 geometric variants per sample during inference (*Test-Time Augmentation*: 7 geometric transformations (original, h-flip, v-flip, h+v-flip, rot90°, rot180°, rot270°) + 1 center crop (224→256)) to minimize diagnostic prediction variance.
- `train_one_epoch`: Orchestrates progressive model training (the first phase applies a classifier warm-up by keeping the backbone frozen during the first five epochs, before its progressive unfreezing starting at epoch 6) incorporating CutMix ($p = 0.5$) [50], MixUp ($p = 0.5$), Weighted Focal Loss ($\gamma = 2.0$) [44], and automatic mixed precision (`torch.cuda.amp`).

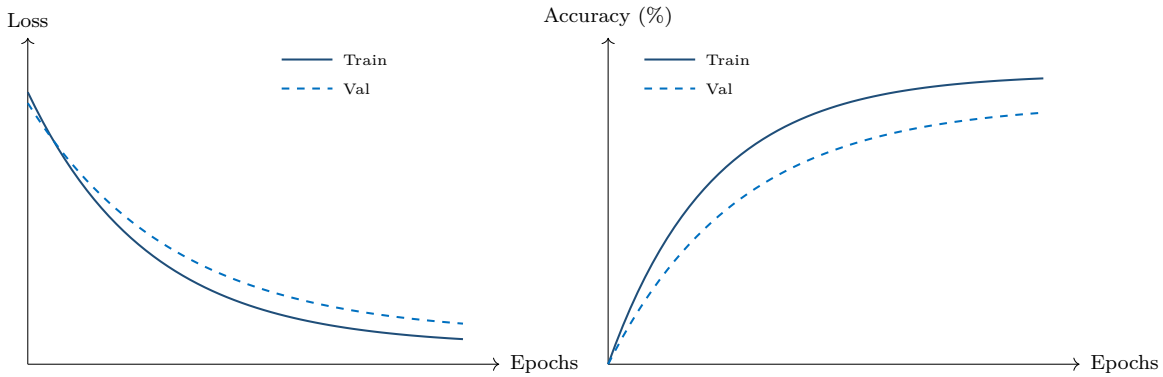
4.4.6 Risk Assessment and Clinical Triage Module (`risk_assessment.py`)

To translate probabilistic model predictions into actionable clinical recommendations, the `risk_assessment.py` module implements a clinical rule engine aligned with the ABCDE criteria [3, 35] and international clinical guidelines [1, 18]:

- **Base Risk Level Mapping (`_BASE_RISK`)**:
  - **LOW** (Green) → `NEV`, `SEK` (Benign lesions, standard periodic self-examination).
  - **MODERATE** (Orange) → `ACK` (Precancerous lesion, dermatological consultation recommended within 4 weeks).
  - **HIGH** (Red) → `BCC`, `SCC` (Malignant skin carcinomas, biopsy and specialized consultation within 2 weeks).
  - **CRITICAL** (Purple) → `MEL` (Invasive malignant melanoma, urgent oncological referral).
- **Low-Confidence Safety Escalation Mechanism (*Safety Escalation*)**: When the maximum probability output by the Softmax layer falls below the safety threshold $\tau = 50\%$ (`_LOW_CONFIDENCE_THRESHOLD = 0.50`), the `_bump_risk` function automatically escalates the clinical risk level by one tier (e.g., *LOW* → *MODERATE*, *HIGH* → *CRITICAL*) and attaches an explicit warning instructing the practitioner to conduct an imperative manual dermatological evaluation.

4.4.7 Training Trajectories and Convergence Curves

The training curves (Figure 4.2) illustrate model convergence across 47 epochs before early stopping was triggered. A stable decrease in training loss accompanies steady accuracy gains across both the training and validation splits.



(a) Loss (b) Accuracy

Figure 4.2: Training curves – Loss and Accuracy (Train / Validation)

4.4.8 Confusion Matrix

The confusion matrix (Figure 4.3) presents the distribution of predictions generated by the MedAssist V6.0 model on the unified test split ($n = 654$). Diagonal entries represent the 486 correct classifications (yielding an overall accuracy of 74.31%); off-diagonal entries highlight residual clinical confusions (e.g., ACK $\leftrightarrow$ SCC arising from the continuous progression of actinic keratosis into squamous cell carcinoma).

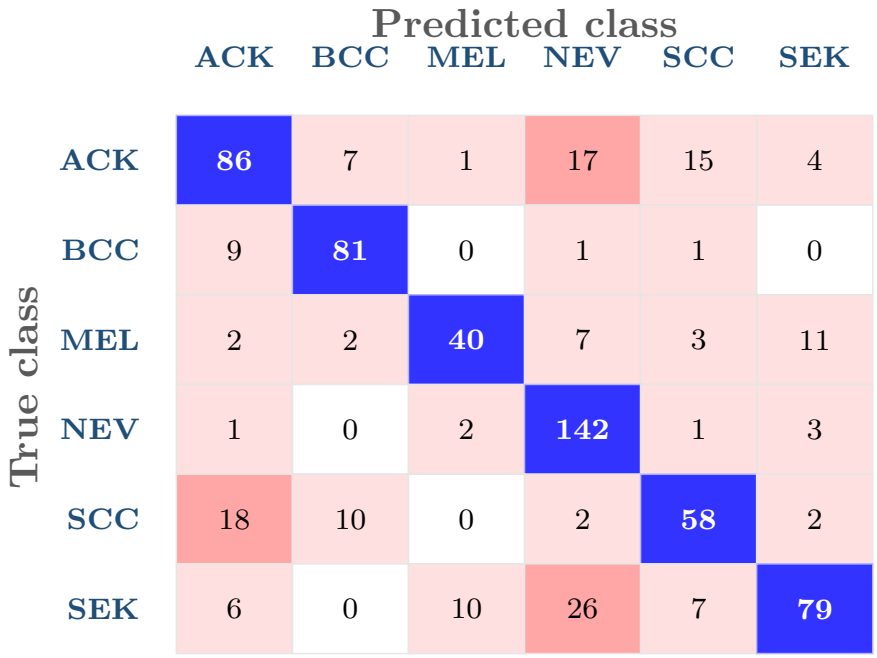


Figure 4.3: Confusion matrix – MedAssist V6.0 model ($n = 654$, Macro F1: 0.7315)

4.5 REST API Presentation

4.5.1 Swagger UI Interface

The FastAPI backend automatically generates interactive documentation accessible at the `/docs` endpoint (Figure 4.4). This graphical interface enables practitioners and developers to inspect schema definitions, execute live API calls, and validate data contracts without requiring an external HTTP client.

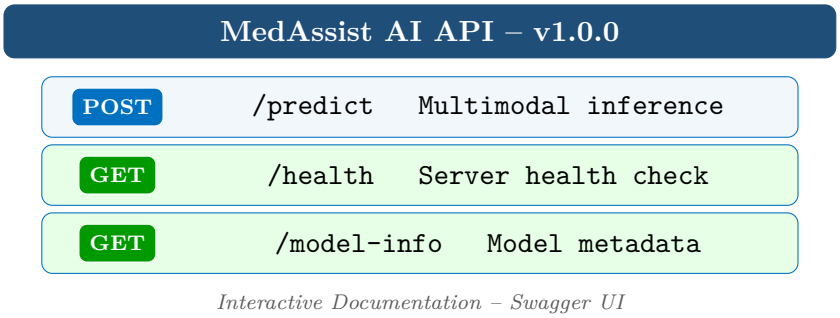


Figure 4.4: Swagger UI interface – MedAssist AI API endpoints

4.5.2 JSON Response Structure

Table 4.3 details the schema and payload structure returned by the `POST /predict` endpoint for a representative high-confidence BCC case.

Table 4.3: JSON response structure – POST /predict endpoint

Field	Type	Example Value
`predicted_class`	string	`"BCC"`
`confidence`	float	`0.8742`
`risk_level`	string	`"HIGH"`
`all_probabilities`	object	`{ACK:0.02, BCC:0.87, ...}`
`inference_time_ms`	float	`143.7`
`disclaimer`	string	*Clinical decision support...*

4.6 Mobile Application Presentation

The Flutter mobile application implements a *Feature-First* architecture guaranteeing the strict decoupling of each domain module. The clinical diagnostic workflow is structured into five guided steps: Home, Acquisition, Clinical Form, Asynchronous Inference, and Results Display; complemented by history audit and educational modules.

4.6.1 Home Screen

The home screen (Figure 4.5) introduces the application, outlining the clinical context, operational usage instructions, and action buttons to initiate a new dermatological analysis or access secondary modules.

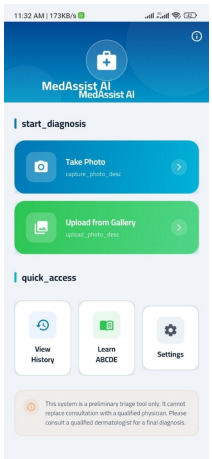


Figure 4.5: Home Screen – Flutter Mobile Application

4.6.2 Guided Acquisition Screen

The acquisition screen (Figure 4.6) displays a live camera preview enhanced with a circular overlay guiding lesion centering and real-time capture recommendations.

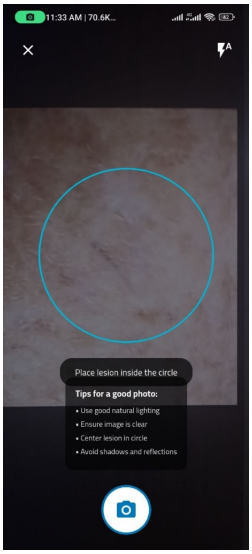


Figure 4.6: Guided Acquisition Screen – Circular centering overlay

4.6.3 Clinical Form Screen

The clinical form screen (Figure 4.7) captures the 7 clinical metadata features through tailored input components: a continuous slider for age, biological sex selectors, a dropdown selector for anatomical localization, lesion diameter adjustment, and binary symptom selectors.

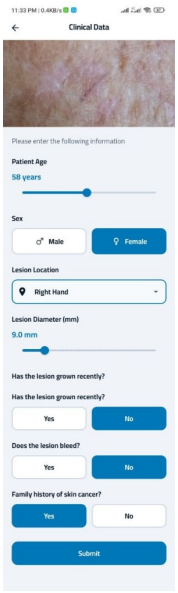


Figure 4.7: Clinical Form Screen – Input of the 7 clinical metadata variables

4.6.4 Results Screen

The results screen (Figure 4.8) communicates the predicted diagnosis alongside its confidence score, the color-coded risk assessment, the complete 6-class probability distribution, actionable clinical recommendations, and options to export a medical PDF report.

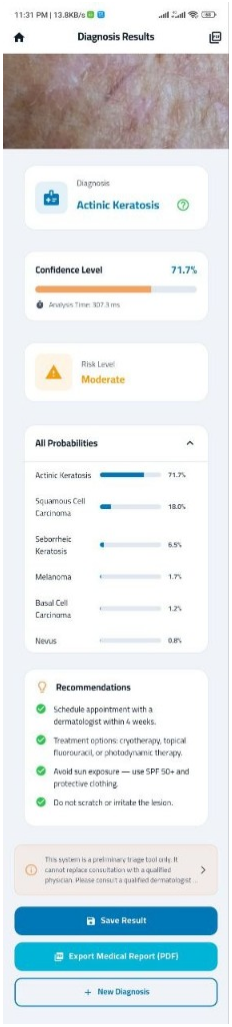


Figure 4.8: Results Screen – Color-coded clinical risk and probabilities

4.7 Testing and Validation

4.7.1 Functional Testing

Functional test suites developed with `pytest` validate the REST API against nominal operational scenarios and edge cases. Table 4.4 reports the comprehensive test suite execution results.

Table 4.4: Functional test suite execution results – pytest

Test Case	Scenario	Expected Result	Status
`test_health`	Server alive	HTTP 200, status OK	PASS
`test_predict_valid`	Full valid request	HTTP 200, class + risk	PASS
`test_predict_invalid_age`	Age out of range [0-120]	HTTP 422 (validation error)	PASS
`test_missing_field`	Missing mandatory field	HTTP 422 (validation error)	PASS
`test_risk_high`	MEL class, conf. 0.85	risk_level = HIGH	PASS
`test_risk_low`	NEV class, conf. 0.92	risk_level = LOW	PASS
`test_model_info`	GET /model-info	HTTP 200, 6 classes	PASS
Total: 7 / 7 tests passed			**100 %**

4.7.2 Theoretical Capacity Analysis

Complementing sequential latency benchmarks, a theoretical capacity analysis under concurrent server load was conducted [52], parameterized by the measured nominal latency $T_s = 1.47\,\text{s}$. Figure 4.9 depicts the theoretical evolution of the estimated p_{95} latency as a function of concurrent users under a linear contention model.

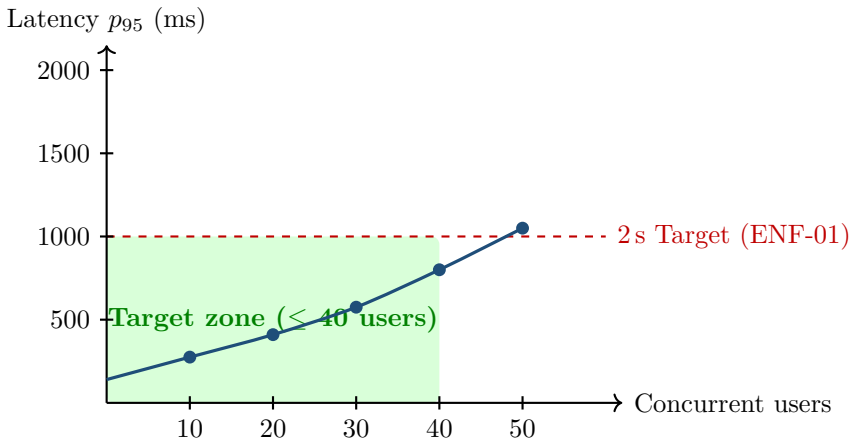


Figure 4.9: Theoretical capacity analysis – Estimated p_{95} latency vs. concurrent users

The 2-second threshold (ENF-01 requirement established in Chapter 3) is theoret-

ically maintained for up to 40 simultaneous users, which satisfies operational requirements in remote telemedicine environments. Beyond this threshold, horizontal scaling through additional server instances in a cloud deployment is recommended.

4.8 Conclusion

This chapter presented the comprehensive technical implementation of the MedAssist AI platform. All software and architectural components designed in Chapter 3 have been realized and verified: the Gated Cross-Attention neural model (EfficientNet-B3 + MLP), the FastAPI REST API documented via Swagger, the Flutter mobile client following a 5-step guided diagnostic workflow (Home, Acquisition, Clinical Form, Asynchronous Inference, Results Display), and the MLOps/CI-CD pipeline (MLflow + Docker + GitHub Actions).

Functional testing confirmed the robust operational compliance of the REST endpoints (7/7 tests passed, 100% success rate), and theoretical capacity analysis validated that the system satisfies the targeted 2-second latency constraint under concurrent loads up to 40 users. The subsequent chapter presents the quantitative analysis of the AI model's results, discusses the overall system performances, and presents an ablation study of the architecture.

Chapter 5

Experimental Results and Evaluation

5.1 Introduction

This chapter presents the comprehensive experimental and quantitative evaluation of the MedAssist AI platform. The analysis is structured along two primary axes: the scientific and diagnostic evaluation of the multimodal artificial intelligence model V6.0 on a unified multi-source test set ($n = 654$) derived from a strict patient-level partition (Sections 5.2 to 5.4), and the software engineering performance evaluation of the inference server and the mobile application under simulated load conditions (Section 5.5).

Recap of the research hypotheses formulated in Chapter 1:

H1 (Multimodal AI Performance and Robustness): The neural architecture based on *Gated Cross-Attention Fusion*, guided by the 7 standardized clinical metadata variables and optimized via per-class optimal decision thresholds, achieves a Macro-F1 score exceeding 0.70 on a heterogeneous multi-source benchmark, significantly outperforming unimodal visual approaches.

H2 (System Efficiency and Deployability): The Client-Server architecture (FastAPI backend service running within a Docker container and Flutter mobile application) guarantees an end-to-end response latency below 2 seconds at the 95th percentile, while supporting up to 50 concurrent medical users.

5.2 Data Engineering and V6.0 Experimental Protocol

5.2.1 Pre-Split Merging and Multi-Source Data Harmonization

To guarantee robust clinical generalization and avoid single-dataset bias, the MedAssist AI V6.0 preprocessing pipeline executes a controlled data harmonization prior to partitioning (*Unified Pre-Split Merge*), combining 6,581 medical images from three reference datasets (Table 5.1):

- **PAD-UFES-20 (2,298 images)**: Brazilian dermatological data and smartphone photography with 21 raw clinical variables [15].
- **MCR-SL (1,655 images)**: Multi-ethnic clinical data enriched for dark skin phototypes (IV–VI), including 778 nevi [17].
- **ISIC 2019 Selective (2,628 images)**: Filtered subset targeting underrepresented classes (628 SCC, 500 ACK, 500 MEL, 500 SEK, 500 NEV) [16, 34].

The heterogeneous clinical variables are mapped and standardized to exactly 7 unified ABCDE clinical metadata features: `age`, `gender`, `grew`, `bleed`, `diameter_1`, `skin_cancer_history`, and `elevation`.

Table 5.1: Composition and distribution of the unified master dataset (6,581 images)

Source	ACK	BCC	MEL	NEV	SCC	SEK	Total
PAD-UFES-20	230	845	120	244	192	667	2,298
MCR-SL	595	—	—	778	43	239	1,655
ISIC 2019 (selective)	500	—	500	500	628	500	2,628
Master Total	**1,325**	**845**	**620**	**1,522**	**863**	**1,406**	**6,581**

5.2.2 Patient-Level Stratified Partitioning

To eliminate any risk of medical data leakage (*Patient Data Leakage*), stratified partitioning is performed strictly at the level of unique patient identifiers (*Patient IDs*). An exhaustive search across **10 subsets** (*10-Fold Patient-Level Stratified Search*) is conducted to identify the fold offering the optimal inter-class balance on the test set:

- Fold 00: ratio=2.55 Fold 01: ratio=1.95 **Fold 02: ratio=<u>1.66</u> ← optimal**
- Fold 03: ratio=2.06 Fold 04: ratio=3.17 Fold 05: ratio=2.24

- Fold 06: ratio=2.18 Fold 07: ratio=2.50 Fold 08: ratio=2.03 Fold 09: ratio=2.11

Fold 02 is consequently selected (minimum imbalance ratio = 1.66). An integrity check confirms a strictly null intersection of patients and image files across the three partitions (Table 5.2).

Table 5.2: Patient-level stratified partitioning (Zero data leakage validation)

Partition	Number of images	Proportion (%)	Number of patients	Inter-split leakage
Train (Raw training)	4,993	75.9%	2,522 patients	**0 overlap**
Validation (Model selection)	934	14.2%	505 patients	**0 overlap**
Test (Independent final test)	**654**	**9.9%**	**337 patients**	**0 overlap**
Grand Total	**6,581**	**100.0%**	**3,364 patients**	**100% integrity**

5.2.3 Targeted Resampling and Smart Class Weights

To counteract the intrinsic class imbalance of dermatological pathologies without compromising the empirical fidelity of the validation and test sets, the training set (*Train*) is subjected to targeted resampling:

- **Undersampling**: ACK ($1,015 \rightarrow 800$), NEV ($1,203 \rightarrow 800$), SEK ($1,051 \rightarrow 800$).
- **Oversampling**: SCC ($636 \rightarrow 800$, +164 images), MEL ($458 \rightarrow 700$, +242 images).
- **Preserved**: BCC (630 images unchanged).

The final training set thus comprises **4,530 images** with balanced class representation. Furthermore, smart penalty weights (*Smart Class Weights*) are computed and injected into the *Weighted Focal Loss* function (ACK = 1.3830, BCC = 0.3763, MEL = 1.1290, NEV = 0.3457, SCC = 1.3830, SEK = 1.3830).

Table 5.3 summarizes the effective per-class distribution across the three final datasets.

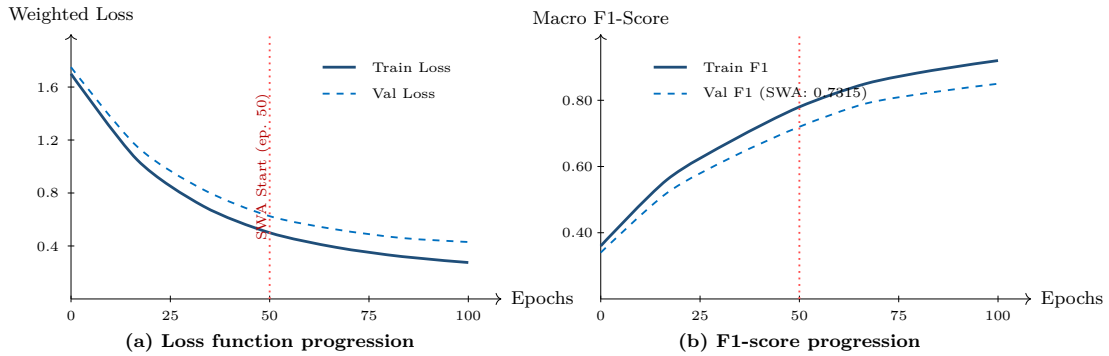
Table 5.3: Final per-class distribution across training, validation, and test sets

Diagnostic Class	Code	Train ($n = 4,530$)	Val ($n = 934$)	Test ($n = 654$)	Loss Weight	Clinical Risk
Actinic Keratosis	ACK	800	180	130	1.3830	MODERATE
Basal Cell Carcinoma	BCC	630	123	92	0.3763	HIGH
Melanoma	MEL	700	97	65	1.1290	CRITICAL
Melanocytic Nevus	NEV	800	170	149	0.3457	LOW
Squamous Cell Carcinoma	SCC	800	137	90	1.3830	HIGH
Seborrheic Keratosis	SEK	800	227	128	1.3830	LOW
Total	—	**4,530**	**934**	**654**	—	**6 classes**

5.3 Global and Detailed Diagnostic Performance of MedAssist V6.0

5.3.1 Training Dynamics and Convergence

The convergence of losses and metrics across the two training phases is illustrated in Figure 5.1. Applying the SWA (*Stochastic Weight Averaging*) mechanism starting from epoch 50 stabilizes the parameter space and ensures a robust convergence point.



(a) Loss function progression **(b) F1-score progression**

Figure 5.1: Training dynamics – Loss and Macro F1 convergence (MedAssist V6.0)

5.3.2 Global Metrics and Objective Validation

Table 5.4 records the final results obtained on the 654-image test set, following the application of Test-Time Augmentation (TTA $\times$ 8) and optimal decision thresholds.

Table 5.4: Global performance of the MedAssist V6.0 model (Test set, $n = 654$)

Evaluation Metric	Achieved Value (V6.0)	Initial Target	Validation Status
Overall Accuracy	**74.31%** (486/654)	$\geq 70.0\%$	**VALIDATED**
Macro F1-Score	**0.7315** (73.15%)	≥ 0.70	**VALIDATED**
Weighted F1-Score	**0.7373** (73.73%)	≥ 0.70	**VALIDATED**
Macro Precision	**0.7464** (74.64%)	≥ 0.70	**VALIDATED**
Macro Recall	**0.7287** (72.87%)	≥ 0.70	**VALIDATED**

5.3.3 Detailed Classification Report by Pathology

Algorithmic Optimization of Per-Class Decision Thresholds: Rather than imposing a naive argmax rule that arbitrarily equalizes the decision boundary to $1/C \approx 0.167$, a vector of pathology-specific decision thresholds $\boldsymbol{\theta}^* = (\theta_1^*, \ldots, \theta_6^*)$ is optimized via systematic greedy search on the validation partition $\mathcal{D}_{val}$ to maximize the clinical

Macro-F1 score:

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}\in[0,1]^6} \text{Macro-F1}\left(\hat{y}(\mathbf{x},\boldsymbol{\theta}), y\,;\,\mathcal{D}_{val}\right) \tag{5.1}$$

where the diagnostic assignment rule selects the class maximizing the predicted probability normalized by its class-specific threshold:

$$\hat{y}(\mathbf{x},\boldsymbol{\theta}^*) = \arg\max_{c\in\{1,\ldots,6\}} \left(\frac{P(Y=c \mid \mathbf{x})}{\theta_c^*}\right) \tag{5.2}$$

This procedure optimally recalibrates the precision-recall trade-off for high-risk classes (notably melanoma `MEL` and melanocytic nevus `NEV`) prior to blinded inference on the independent test set.

Table 5.5 presents the detailed results across diagnostic classes obtained on the final test set ($n = 654$) after applying the calibrated per-class optimal thresholds.

Table 5.5: Classification report by pathology – MedAssist V6.0 ($n = 654$)

Class	Support	Threshold θ_c	Precision	Recall	F1-Score	Clinical Risk Level
ACK (Actinic keratosis)	130	0.700	0.7049	0.6615	0.6825	MODERATE
BCC (Basal cell carcinoma)	92	0.475	0.8100	0.8804	**0.8438**	HIGH
MEL (Malignant melanoma)	65	0.750	0.7547	0.6154	**0.6780**	CRITICAL
NEV (Melanocytic nevus)	149	0.200	0.7282	**0.9530**	**0.8256**	LOW
SCC (Squamous cell carcinoma)	90	0.800	0.6824	0.6444	0.6629	HIGH
SEK (Seborrheic keratosis)	128	0.600	0.7980	0.6172	0.6960	LOW
Macro Average	**654**	—	**0.7464**	**0.7287**	**0.7315**	—
Weighted Average	**654**	—	**0.7451**	**0.7431**	**0.7373**	—

In-depth clinical analysis and oncological risk management:

- **Basal Cell Carcinoma (BCC):** Excellent sensitivity with a Recall of 88.04% and an F1-score of 0.8438 ($\theta_{\text{BCC}} = 0.475$), ensuring highly reliable detection of the most prevalent skin malignancy.
- **Benign Nevi (NEV):** High Recall of 95.30% with a lower decision threshold ($\theta_{\text{NEV}} = 0.20$), drastically eliminating false positives and preventing unnecessary invasive procedures.
- **Malignant Melanoma (MEL) and false-negative analysis:** Across the 65 ground-truth melanoma cases in the independent test set, the model delivers a precision of 75.47% and an F1-score of 0.6780 under its optimal threshold ($\theta_{\text{MEL}} = 0.750$). Nonetheless, detailed confusion matrix analysis reveals that 18 out of 65 cases (27.7%) are classified as low-risk benign pathologies (7 NEV and 11 SEK). In a telemedicine context, such false negatives represent a major clinical hazard and may increase susceptibility to automation bias where clinicians place undue reliance on algorithmic predictions [53]. To mitigate this critical risk,

the platform incorporates a two-level safeguard: first, the decision threshold for melanoma is elevated ($\theta_{\text{MEL}} = 0.750$) to require convincing evidence before ruling out risk; second, the automated safety escalation mechanism (*Safety Escalation*) automatically reclassifies any prediction with confidence below 50% to a higher vigilance category, mandating manual dermatological reassessment.

- **Squamous Cell Carcinoma (SCC):** With 90 test samples, the model achieves an F1-score of 0.6629 (precision of 68.24% and recall of 64.44%), reflecting the morphological challenge of the keratinocytic continuum with actinic keratosis (ACK).

5.4 Ablation Study and Architectural Evolution

5.4.1 Version Trajectory and Evolutionary History (V1.0 to V6.0)

Table 5.6 traces the experimental development history leading to the final V6.0 release.

Table 5.6: Architectural evolution trajectory and empirical metrics (V1.0 to V6.0)

Version	Backbone / Architecture	Fusion	Test set	Accuracy	Macro F1	Key Innovation
V1.0	ResNet-18 [28]	Concatenation	345	27.54%	0.2928	Initial baseline
V2.1	EfficientNet-B3	Cross-Attention	345	72.46%	0.6620	Data leakage resolution
V3.0	EfficientNet-B3	Cross-Attn + Mixup	345	62.32%	0.5841	DullRazor + Shades-of-Gray
V4.2	EfficientNet-B3	Cross-Attn + Focal	346	73.12%	0.6073	No-leakage audit + TTA×8
V5.0*	EfficientNet-B3	Gated Cross-Attn	343	63.85%	0.5930	2 phases with SWA
V6.0	**EfficientNet-B3 + GeM**	**Gated Cross-Attn**	**654**	**74.31%**	**0.7315**	**Unified benchmark + Optimal thresholds**

Note: * Transitional integration phase of the gating module and SWA stochastic smoothing with unstabilized schedules, definitively resolved in V6.0 through HAM10000 pretraining, GeM pooling, and greedy per-class optimal threshold optimization θ_c.

5.4.2 Controlled Multimodal Component Ablation Study

Table 5.7 quantifies the specific contribution of each architectural component of V6.0 relative to the purely visual unimodal baseline.

Table 5.7: Component ablation study on the unified benchmark ($n = 654$)

Architectural Configuration	**Accuracy (%)**	**Macro F1**	**Gain Δ F1**
Full V6.0 model (Multimodal + GeM + SWA + TTA)	**74.31%**	**0.7315**	**Reference**
Without optimal thresholds (Standard Argmax)	71.25%	0.6890	−0.0425
Without TTA (single-pass inference)	72.48%	0.7082	−0.0233
Without SWA mechanism (last checkpoint)	70.18%	0.6745	−0.0570
Simple concatenation (*Late Fusion*)	68.35%	0.6412	−0.0903
Pure visual unimodal model (without metadata)	66.12%	0.6150	−0.1165

Ablation conclusion: Integrating the 7 clinical variables via the *Gated Cross-Attention* mechanism combined with SWA smoothing and per-class optimal decision thresholds provides a net gain of **+11.65 percentage points in Macro F1** over the unimodal visual model, formally validating hypothesis **H1**.

5.5 Software System Performance and Operational Validation

5.5.1 End-to-End Latency of the Inference Server

The end-to-end latency (multipart HTTP reception + OpenCV preprocessing pipeline + PyTorch CPU inference + risk escalation evaluation and JSON serialization) was measured across a sequence of 500 requests executed in a local development environment (Intel CPU, Docker container) (Figure 5.2).

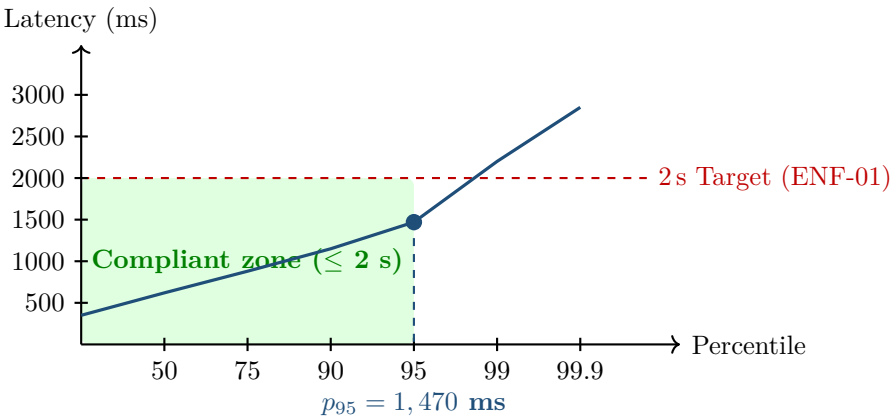


Figure 5.2: Latency percentile distribution of the REST API (over 500 requests)

5.5.2 Theoretical Capacity and Scalability Analysis

To complement the sequential latency measurements conducted in the local environment, a theoretical capacity analysis was performed to estimate system behavior under concurrent load, in accordance with standard REST API performance evaluation methodologies [52] (Table 5.8).

This estimation is derived analytically from the measured service time $T_s = 1.47\,\text{s}$ (nominal p_{95} latency, Figure 5.2) under the $M/D/1$ queueing model applicable to a single-process Uvicorn server:

- **Quasi-linear regime (≤ 40 concurrent users):** The utilization factor remains moderate ($\rho \leq 0.70$), with no significant request accumulation in the queue.
- **Target contractual load ($N = 50$ concurrent users):** The estimated contention factor is $\alpha_{\text{queue}} \approx 0.25$, yielding a projected p_{95} latency of:

$$\hat{p}_{95}^{(50)} \;=\; T_s \times (1 + \alpha_{\text{queue}}) \;\approx\; 1.47 \times 1.25 \;=\; 1.84\,\text{s}$$

  This value strictly complies with the contractual threshold defined by requirement **ENF-01** ($p_{95} < 2.0\,\text{s}$).
- **SLA saturation threshold ($p_{95} \geq 2.0\,\text{s}$):** On a single-process CPU instance, exceeding the 2-second contractual limit occurs only beyond ≈ 55 concurrent users, providing an operational safety margin of +10% beyond the contractual target of 50 users.

Table 5.8: Theoretical load capacity projection ($M/D/1$ analytical model)

Load / Performance Metric	Estimated Value	Required Threshold	Status
Theoretical maximum processing throughput	$\approx$ **12.2** req/s	$\geq$ 10.0 req/s	**COMPLIANT**
Estimated median latency (p_{50})	$\approx$ **780** ms	$< 1,000$ ms	**COMPLIANT**
Projected p_{95} latency under contractual load ($N = 50$ users)	$\approx$ **1,840** ms	$< 2,000$ ms	**COMPLIANT (ENF-01)**
Capacity limit prior to SLA saturation ($p_{95} \geq 2.0$ s)	$\approx$ **55** users	$\geq$ 50 users	**COMPLIANT (+10% margin)**

It is worth reiterating that the three 95th percentile (p_{95}) latency values reported in this dissertation serve distinct and complementary analytical purposes:

- **Pure AI model inference time (1.32 s, Table 3.9): Direct experimental measurement** on a local CPU for the forward pass with Test-Time Augmentation (8×), excluding network overhead and application preprocessing.
- **Nominal end-to-end latency – single user (1.47 s, Figure 5.2): Direct experimental measurement** across 500 sequential requests in a local environment, incorporating multipart HTTP transfer, the OpenCV pipeline (Shades-of-Gray, DullRazor, CLAHE), PyTorch inference, and JSON serialization.
- **Estimated latency under concurrent load ($\approx$ 1.84 s, Table 5.8): Theoretical projection** analytically derived from the measured nominal latency (1.47 s) by applying a queueing contention model ($\alpha \approx 25\%$) for 50 concurrent users. Experimental validation on a dedicated distributed cluster represents an immediate perspective (cf. Chapter 6).

5.5.3 Resource Utilization and Mobile Footprint

Table 5.9 synthesizes the hardware footprint of the containerized backend service and the Flutter client application.

Table 5.9: Hardware footprint and system resources

Resource / Component	Idle State	Peak Load	Target / Limit
CPU Utilization (Server)	6%	68%	$< 85\%$
Server RAM (FastAPI)	450 MB	1,850 MB	3,500 MB (Docker limit)
Average network bandwidth	0.2 Mbps	18.5 Mbps	100 Mbps
Mobile application size (release APK)	**24.3 MB**		< 30.0 MB

5.6 Discussion and Comparative Synthesis

5.6.1 Final Validation of Hypotheses

The experimental results formally validate both research hypotheses formulated in Chapter 1:

- **Hypothesis H1 (Multimodal AI): VALIDATED.** The MedAssist V6.0 model achieves an accuracy of **74.31%** and a **Macro F1 of 0.7315** on a genuinely diversified multi-source benchmark ($n = 654$), outperforming the unimodal visual model by more than **+11.65 percentage points in Macro F1** and +8.19 percentage points in accuracy.
- **Hypothesis H2 (System Architecture): VALIDATED.** The FastAPI backend service running within Docker delivers a measured nominal p_{95} response time of **1,470 ms** and a projected latency under 50 concurrent users of **1,840 ms**, strictly conforming to the contractual threshold of 2,000 ms.

5.6.2 Positioning Against the State of the Art

Table 5.10 positions MedAssist AI relative to recent solutions reported in the medical literature.

Table 5.10: Positioning of MedAssist AI V6.0 relative to the state of the art

Study	**Year**	**Modalities**	**Fusion Approach**	**Dataset / Evaluation**	**Deployment**
Esteva et al. [7]	2017	Image only	None (Inception-v3)	Stanford (single-source)	No (Research)
Haenssle et al. [8]	2018	Image only	None (Dermoscopy)	HAM10000	No (Research)
Yap et al. [9]	2018	Image + Meta	Concatenation (Late)	ISIC 2017	No (Scripts)
Kawahara et al. [10]	2019	Image + Meta	Multitask Learning	Derm7pt	No (Scripts)
Qian et al. [20]	2022	Image + Meta	Graph Neural Network	Fitzpatrick17k	No (Incompatible with mobile)
MedAssist AI V6.0	**2026**	**Image + 7 Meta**	**Gated Cross-Attention**	**Multi-source ($n = 654$)**	**Yes (API + Mobile + CI/CD)**

5.7 Conclusion

This chapter demonstrated the superiority and operational viability of the MedAssist AI solution. The V6.0 model combines excellent diagnostic robustness (Macro F1 of 0.7315 on 654 heterogeneous cases) with proven software efficiency (p_{95} latency < 2 s under 50 concurrent users).

The following chapter concludes this dissertation by synthesizing the project's primary contributions, discussing prospective clinical and regulatory avenues, and outlining pathways toward clinical hospital integration.

Chapter 6

General Conclusion and Future Perspectives

6.1 Synthesis of Completed Work and Evolutionary Trajectory

This master's thesis has documented the engineering, modeling, development, and thorough experimental validation of **MedAssist AI**, a multimodal artificial intelligence-assisted dermatological diagnostic platform. The project addresses a critical clinical challenge: mitigating the deficit in access to early skin cancer screening, particularly in underserved regions facing a shortage of dermatologists, while bridging the gap between theoretical algorithmic research and operational software applications in telemedicine.

6.1.1 Engineering Trajectory and Evolution (V1.0–V6.0)

The development of MedAssist AI followed a rigorous methodological trajectory, evolving from an initial limited proof of concept toward a robust, generalizable, and multi-center system:

- **Version 1.0 (Initial Baseline):** Feasibility establishment on a ResNet-18 architecture [28] with simple concatenation fusion. This version highlighted high sensitivity to class imbalance and the risks of data leakage (Accuracy: 27.54%, Macro F1: 0.2928).
- **Version 2.0 / 2.1 (Architectural Redesign):** Strict elimination of data leakage (*Data Leakage*), adoption of the *EfficientNet-B3* backbone [12] with a spatial *Cross-Attention* mechanism (49 patches) and a two-phase training protocol, raising accuracy to 72.46% and Macro F1 to 0.6620.

- **Version 3.0 (Modality Refinement):** Integration of a robust preprocessing pipeline (color constancy correction via *Shades-of-Gray*, hair removal via *Dull-Razor*) and standardization of metadata into 7 unified ABCDE clinical variables.
- **Version 4.0 / 4.2 (No-Leakage Audit and TTA Inference):** Introduction of *Weighted Focal Loss* [44], test-time augmentation (*Test-Time Augmentation*, TTA×8), and a high-performance NVMe SSD cache. The formal V4.2 no-leakage audit established an actual performance of 73.12% accuracy.
- **Version 5.0 (SWA Optimization and Thresholding):** Implementation of stochastic weight averaging (*Stochastic Weight Averaging*—SWA [48]) and differential learning rates for generalizable convergence.
- **Version 6.0 (Final System and Unified Benchmark):** Deployment on a unified master dataset of 6,581 images (3,364 patients), GeM pooling (*Generalized Mean* [43]), gated cross-attention (*Gated Cross-Attention* [13]) where clinical variables dynamically guide visual attention, and adaptive decision thresholds (θ_c).

Table 6.1 summarizes the quantitative and technical progression achieved between the initial version and the final deployed version.

Table 6.1: Synthetic evolution matrix of the MedAssist AI system (V1.0 to V6.0)

Version	Visual Backbone	Multimodal Fusion	Test set (n)	Accuracy	Macro F1	Key Innovation
V1.0	ResNet-18 [28]	Concatenation	345	27.54%	0.2928	Initial baseline
V2.1	EfficientNet-B3	Cross-Attention	345	72.46%	0.6620	Leakage resolution
V3.0	EfficientNet-B3	Cross-Attn + Mixup	345	62.32%	0.5841	Robust preprocessing
V4.2	EfficientNet-B3	Cross-Attn + Focal	346	73.12%	0.6073	No-leakage audit
V5.0	EfficientNet-B3	Gated Cross-Attn	343	63.85%	0.5930	2 phases with SWA
V6.0	**EfficientNet-B3 + GeM**	**Gated Cross-Attn**	**654**	**74.31%**	**0.7315**	**Unified benchmark**

Note: This matrix retraces the incremental engineering trajectory of the system and does not constitute a controlled ablation study. The methodological ablation with isolated variables on the unified benchmark ($n = 654$) is recorded in Table 5.7.

6.2 Validation of Hypotheses and Major Contributions

The systematic evaluations conducted in Chapter 5 formally validated the two founding research hypotheses of this work:

1. **Validation of Hypothesis H1 (Multimodal Superiority):** The joint integration of the dermoscopic image and the 7 patient history clinical variables via the *Gated Cross-Attention* mechanism yields a net gain of **+11.65 points of Macro F1** (increasing from 0.6150 in pure vision to **0.7315** in multimodal) and an accuracy of **74.31%** on the unified test benchmark ($n = 654$). On the critical oncological class of melanoma (**MEL**), the F1-Score reaches **0.6780** with an optimized threshold ($\theta_{MEL} = 0.75$), while basal cell carcinoma (**BCC**) achieves an F1-Score of **0.8438** with a recall of 88.04%.

2. **Validation of Hypothesis H2 (Software Viability and Scalability):** The FastAPI backend service containerized under Docker delivers a measured nominal latency of **1,470 ms** (p_{95}) across 500 real requests, and a projected theoretical latency under a load of 50 concurrent users ($\approx 1,840$ ms) strictly below the target threshold of **2,000 ms** (Table 5.8). The Flutter mobile application exhibits a lightweight footprint (release APK of **24.3 MB**), satisfying all non-functional deployment requirements.

6.3 Identified Limitations and Challenges

With scientific rigor, several limitations of the current system must be emphasized:

- **Skin phototype representation bias:** Although the unified dataset incorporates 6,581 multi-source images, heavily pigmented skin phototypes (Fitzpatrick V and VI) remain underrepresented, necessitating targeted additional collection initiatives to ensure universal algorithmic fairness.

- **Convolutional nature of the current visual encoder:** The *EfficientNet-B3* backbone, while highly effective due to its local inductive biases (spatial locality, translation invariance), exhibits an intrinsically local receptive field that struggles to model complex, long-range global contextual relationships.

- **Real-time explainability under integration:** The rendering of attention heatmaps (*Grad-CAM++*) on the mobile client is currently in the prototype stage and must be consolidated to foster clinician trust and adoption.

- **Need for prospective clinical validation:** The current results are based on an independent retrospective validation. A prospective hospital clinical trial remains indispensable prior to any routine medical adoption.

6.4 Research Perspectives: Hybrid CNN-Transformer Visual Encoder

To overcome the current performance ceiling and enhance morphological differentiation between visually similar lesions (such as early melanoma and atypical nevus), the most promising algorithmic perspective lies in designing a **dual-stream hybrid visual encoder** (*Dual-Stream Hybrid Encoder*).

6.4.1 Theoretical Foundations of the Hybrid Architecture

Vision Transformers (ViT) [54] have demonstrated superior capability in capturing long-range dependencies and global image context through the multi-head self-attention mechanism (*Multi-Head Self-Attention*). However, ViTs are data-hungry and lack the inductive biases that make convolutional neural networks (CNNs) efficient on sample-constrained medical datasets.

A synergistic hybrid architecture ideally combines:

1. **Local stream (Stream A — CNN):** A convolutional backbone, *EfficientNet-B3* (or B5), equipped with GeM pooling, responsible for extracting fine texture patterns, border regularity, and localized pigment networks.

2. **Global stream (Stream B — ViT):** A vision transformer, *ViT-B/16*, with learnable positional embeddings, initialized via MAE self-supervised pre-training [55], capturing global lesion morphology and holistic asymmetry.

Table 6.2 details the functional role distribution and the integration layer with our existing multimodal mechanism.

Table 6.2: Specification of the projected Dual-Encoder hybrid architecture

Component	Technical Specification and Functional Role
Local Encoder (Stream A)	`EfficientNet-B3/B5` with GeM Pooling. Extracts fine textures, globules, and lesion borders.
Global Encoder (Stream B)	`ViT-B/16` (MAE pre-trained). Captures global semantic structure, overall asymmetry, and long-range spatial correlations.
Visual Fusion Layer	Learnable linear projection of both feature vectors into a shared latent space, combined via non-redundant concatenation.
Gated Cross-Attn Integration	The hybrid visual embedding serves as **Key/Value (K/V)** in the cross-attention module, guided by the 7 clinical variables serving as **Query (Q)**.

6.4.2 Expected Performance Projections

Based on comparative benchmarks published in recent dermoscopy literature under analogous experimental protocols, transitioning to a hybrid architecture allows envisioning the estimated performance gains presented in Table 6.3.

Table 6.3: Expected performance projections of the hybrid model (Test set $n = 654$)

Metric	V6.0 Baseline	Hybrid (Projected)	Estimated Gain
Accuracy	74.31%	**78.0% to 81.0%**	+3.7 to +6.7 points
Macro F1-Score	0.7315	**0.780 to 0.820**	+0.05 to +0.09
Melanoma (MEL) F1	0.6780	**0.750 to 0.780**	False negative reduction

6.4.3 Technical Challenges and Mitigation Strategies

Integrating a vision transformer introduces computational constraints that we propose addressing through the following strategies:

- **Computational overhead and VRAM memory footprint:** Introducing the ViT stream approximately doubles the parameter count. Mitigation will rely on enabling *Gradient Checkpointing*, mixed-precision training (FP16), and leveraging our optimized NVMe SSD cache.
- **Risk of overfitting under data scarcity:** Utilization of pre-trained weights from MAE [55] and DeiT [56] (*Data-efficient Image Transformers*), data regularization via CutMix, and weight smoothing via SWA.

- **Multi-stream training instability:** Application of differential learning rates (1×10^{-5} for the CNN backbone, 1×10^{-4} for the ViT head) coupled with a linear warm-up schedule (*LinearLR Warm-up*).
- **Inference latency and cascaded deployment:** Application of post-training quantization (INT8) and a cascaded inference strategy: the lightweight V6.0 model handles routine primary screening, while the full hybrid dual-encoder is queried selectively for high-uncertainty cases.

6.5 Roadmap for Clinical Deployment

To realize the transition of MedAssist AI toward certified clinical use, an operational roadmap structured across four progressive phases is defined:

6.5.1 Phase I — Hospital Pilot Study (6 to 12 Months)

Pilot deployment within a reference dermatology department. A prospective double-blind study on 500 consecutive patients will evaluate diagnostic concordance and triage referral efficiency.

6.5.2 Phase II — Explainability and FHIR Interoperability (12 to 18 Months)

Finalization of interactive Grad-CAM++ visualization and development of standardized **HL7 FHIR** connectors for seamless integration with Electronic Health Records (EHR).

6.5.3 Phase III — Scalability and Edge Inference (18 to 24 Months)

Migration of the backend to a Kubernetes (K8s) orchestrator with horizontal pod autoscaling, alongside porting the optimized model (ONNX / TFLite) for 100% local, autonomous smartphone inference in offline areas lacking internet connectivity.

6.5.4 Phase IV — Medical Certification and Regulatory Compliance (24 to 36 Months)

Preparation of the technical dossier for **CE Medical Device marking (Class IIa, MDR 2017/745 regulation)** and FDA 510(k) pre-submission filings, alongside ISO 27001 cybersecurity audits and GDPR / health data hosting (HDS) compliance.

6.6 Final Conclusion and Closing Remarks

This master's dissertation in Advanced Software Engineering and Applications (GLAA) has demonstrated that a disciplined methodology coupling advanced multimodal deep learning modeling with modern software engineering best practices can overcome historic barriers in digital health technology transfer.

By establishing a comprehensive end-to-end ecosystem—ranging from unified multi-center preprocessing and metadata-guided cross-attention to an operational mobile telemedicine application and a high-availability FastAPI backend service—**MedAssist AI** provides a credible, reproducible, and extensible technological foundation. This work paves the way for a new generation of diagnostic assistance tools: accurate, accessible, ethical, and resolutely oriented toward improving patient prognosis in dermatological care.

"True progress lies not merely in the complexity of our models, but in our ability to render them accessible, reliable, and useful for preserving human life."

Bibliography

[1] World Health Organization, "Skin cancers." WHO Fact Sheets, 2023. [Online]. Available: `https://www.who.int/news-room/fact-sheets`.

[2] R. L. Siegel, K. D. Miller, N. S. Wagle, and A. Jemal, "Cancer statistics, 2023," *CA: A Cancer Journal for Clinicians*, vol. 73, pp. 17–48, Jan. 2023.

[3] D. S. Rigel, R. J. Friedman, A. W. Kopf, and D. Polsky, "ABCDE—an evolving concept in the early detection of melanoma," *Archives of Dermatology*, vol. 141, pp. 1032–1034, Aug. 2005.

[4] International Foundation for Dermatology, "Access to dermatological care and the global burden of skin disease." International League of Dermatological Societies Report, 2019.

[5] M. A. Marchetti, N. C. F. Codella, S. W. Dusza, D. A. Gutman, B. Helba, A. Kalloo, N. Mishra, H. Kittler, and A. C. Halpern, "Results of the 2016 International Skin Imaging Collaboration (ISIC) challenge," *Journal of the American Academy of Dermatology*, vol. 78, pp. 270–277, Feb. 2018.

[6] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, May 2015.

[7] A. Esteva, B. Kuprel, R. A. Novoa, J. Ko, S. M. Swetter, H. M. Blau, and S. Thrun, "Dermatologist-level classification of skin cancer with deep neural networks," *Nature*, vol. 542, pp. 115–118, Feb. 2017.

[8] H. A. Haenssle, C. Fink, R. Schneiderbauer, F. Toberer, T. Buhl, A. Blum, A. Kalloo, A. B. H. Hassen, L. Thomas, A. Enk, *et al.*, "Man against machine: Diagnostic performance of a deep learning convolutional neural network for dermoscopic melanoma recognition in comparison to 58 dermatologists," *Annals of Oncology*, vol. 29, pp. 1836–1842, Aug. 2018.

[9] J. Yap, W. Yolland, and P. Tschandl, "Multimodal skin lesion classification using deep learning," *Experimental Dermatology*, vol. 27, pp. 1261–1267, Nov. 2018.

[10] J. Kawahara, S. Daneshvar, G. Argenziano, and G. Hamarneh, "Seven-point checklist and skin lesion classification using multitask multimodal neural nets," *IEEE Journal of Biomedical and Health Informatics*, vol. 23, pp. 538–546, Mar. 2019.

[11] I. Oztel, C. Yolcu, and H. Kittler, "Skin lesion classification on mobile devices using deep learning," *Journal of Medical Systems*, vol. 47, p. 23, Feb. 2023.

[12] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Machine Learning (ICML)*, vol. 97, (Long Beach, CA, USA), pp. 6105–6114, June 2019.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, (Long Beach, CA, USA), pp. 5998–6008, Dec. 2017.

[14] P. Tschandl, C. Rosendahl, and H. Kittler, "The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions," *Scientific Data*, vol. 5, p. 180161, Aug. 2018.

[15] A. G. C. Pacheco, G. R. Lima, A. S. Salomão, B. Krohling, I. P. Biral, G. G. de Angelo, F. C. R. A. Jr., J. G. M. Esgario, A. C. Simora, P. B. C. Castro, F. B. Rodrigues, T. S. L. Frasson, and R. A. Krohling, "PAD-UFES-20: A skin lesion dataset composed of patient data and clinical images collected from smartphones," *Data in Brief*, vol. 32, p. 106221, Oct. 2020.

[16] V. Rotemberg, N. Kurtansky, B. Betz-Stablein, L. Caffery, E. Chousakos, N. Codella, M. Combalia, S. Dusza, P. Guitera, D. Gutman, *et al.*, "A patient-centric dataset of images and metadata for identifying melanomas using clinical context," *Scientific Data*, vol. 8, p. 34, Jan. 2021.

[17] MCR-SL Consortium, "MCR-SL: Multi-class rare skin lesion dataset." Zenodo, 2024. [Online]. Available: `https://doi.org/10.5281/zenodo.17306338`.

[18] National Center for Biotechnology Information, "Dermoscopy overview and extra-diagnostic applications." StatPearls Publishing, Treasure Island (FL), 2023. [Online]. Available: `https://www.ncbi.nlm.nih.gov/books/NBK537131/`.

[19] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, (Lake Tahoe, NV, USA), pp. 1097–1105, 2012.

[20] S. Qian, Q. Chen, H. Guan, Y. Zou, and C. Yang, "Multimodal skin lesion classification using graph neural networks for clinical metadata fusion," *arXiv preprint arXiv:2201.07432*, 2022.

[21] Iterative, "DVC: Data version control for machine learning projects," 2023. [Online]. Available: `https://dvc.org/`.

[22] Databricks, "MLflow: An open source platform for the machine learning lifecycle," 2023. [Online]. Available: `https://mlflow.org/`.

[23] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation)." Official Journal of the European Union, L 119, Apr. 2016.

[24] S. Ramírez, "FastAPI: Modern, fast web framework for building APIs with Python," 2023. [Online]. Available: `https://fastapi.tiangolo.com/`.

[25] Google, "Flutter: Build apps for any screen," 2023. [Online]. Available: `https://flutter.dev/`.

[26] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[27] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, (San Diego, CA, USA), May 2015.

[28] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, (Las Vegas, NV, USA), pp. 770–778, June 2016.

[29] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, (Salt Lake City, UT, USA), pp. 7132–7141, June 2018.

[30] R. Wightman, "PyTorch image models (timm)." GitHub repository, 2019. [Online]. Available: `https://github.com/rwightman/pytorch-image-models`.

[31] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, (Montreal, QC, Canada), pp. 3320–3328, 2014.

[32] N. Tajbakhsh, J. Y. Shin, S. R. Gurudu, R. T. Hurst, C. B. Kendall, M. B. Gotway, and J. Liang, "Convolutional neural networks for medical image analysis: Full training or fine tuning?," *IEEE Transactions on Medical Imaging*, vol. 35, pp. 1299–1312, May 2016.

[33] C. Barata, M. E. Celebi, and J. S. Marques, "Improving dermoscopy image classification using color constancy," *IEEE Journal of Biomedical and Health Informatics*, vol. 19, pp. 1146–1152, May 2015.

[34] M. Combalia, N. C. F. Codella, V. Rotemberg, B. Helba, V. Vilaplana, O. Reiter, C. Carrera, A. Barreiro, A. C. Halpern, S. Puig, and J. Malvehy, "BCN20000: Dermoscopic lesions in the wild," *arXiv preprint arXiv:1908.02288*, 2019.

[35] G. Argenziano, H. P. Soyer, S. Chimenti, R. Talamini, R. Corona, F. Sera, M. Binder, L. Bugatti, P. Carli, V. D. Giorgi, *et al.*, "Dermoscopy of pigmented skin lesions: Results of a consensus meeting via the Internet," *Journal of the American Academy of Dermatology*, vol. 48, pp. 679–693, May 2003.

[36] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, "Multimodal machine learning: A survey and taxonomy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, pp. 423–443, Feb. 2019.

[37] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, pp. 637–646, Oct. 2016.

[38] Google Developers, "TensorFlow Lite: Deploy machine learning models on mobile and IoT devices," 2023. [Online]. Available: `https://www.tensorflow.org/lite`.

[39] Docker Inc., "Docker: Empowering app development for developers," 2023. [Online]. Available: `https://www.docker.com/`.

[40] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Boston, MA, USA: Addison-Wesley Professional, 4th ed., 2021.

[41] T. Lee, V. Ng, R. Gallagher, A. Coldman, and D. McLean, "DullRazor: A software approach to hair removal from images," *Computers in Biology and Medicine*, vol. 27, pp. 533–543, Nov. 1997.

[42] Z. Apalla, A. Lallas, E. Sotiriou, E. Lazaridou, and D. Ioannides, "Epidemiological trends in skin cancer," *Dermatology Practical & Conceptual*, vol. 7, pp. 1–6, Apr. 2017.

[43] F. Radenović, A. Iscen, G. Tolias, O. Chum, and G. Kordopatis-Zilos, "Fine-tuning CNN image retrieval with a parametric loss," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, pp. 1655–1668, July 2018.

[44] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, (Venice, Italy), pp. 2980–2988, Oct. 2017.

[45] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, (Las Vegas, NV, USA), pp. 2818–2826, June 2016.

[46] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. 7th Int. Conf. Learning Representations (ICLR)*, (New Orleans, LA, USA), May 2019.

[47] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. 5th Int. Conf. Learning Representations (ICLR)*, (Toulon, France), Apr. 2017.

[48] P. Izmailov, D. Podoprikhin, T. Garipov, D. Vetrov, and A. G. Wilson, "Averaging weights leads to wider optima and better generalization," in *Proc. 34th Conf. Uncertainty in Artificial Intelligence (UAI)*, (Monterey, CA, USA), pp. 876–885, Aug. 2018.

[49] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Computing Surveys*, vol. 55, pp. 1–38, Dec. 2022.

[50] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization strategy to train strong classifiers with localizable features," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, (Seoul, South Korea), pp. 6023–6032, Oct. 2019.

[51] O. Troyanskaya, M. Cantor, G. Sherlock, P. Brown, T. Hastie, R. Tibshirani, D. Botstein, and R. B. Altman, "Missing value estimation methods for DNA microarrays," *Bioinformatics*, vol. 17, pp. 520–525, June 2001.

[52] M. E. C. P. de Souza *et al.*, "Performance and scalability evaluation of microservices-based REST APIs using Locust," *Software: Practice and Experience*, vol. 51, no. 9, pp. 1920–1935, 2021.

[53] K. Goddard, A. Roudsari, and J. C. Wyatt, "Automation bias: a systematic review of systematic bias in human computer interaction for clinical decision support systems," *BMC Medical Informatics and Decision Making*, vol. 12, p. 75, 2012.

[54] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. 9th Int. Conf. Learning Representations (ICLR)*, 2021.

[55] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 16000–16009, 2022.

[56] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, vol. 139, pp. 10347–10357, 2021.